

ДИПЛОМНЫЙ ПРОЕКТ

На тему: Разработка программного средства анализа журналов безопасности операционной системы на рабочей станции

по образовательной программе: 6В06301 «Системы информационной безопасности»

Выполнил

Жалмаганбетов Е.М.

**Руководитель
ст. преподаватель**

Винтерголлер И.Г.

Введение

Во время преддипломной практики в филиале «ТрансТелеКом» (Астана) я смотрел, как растут журналы Windows.

Сотни 4624/4625 за смену в Event Viewer без фильтра почти не читаются.

Очистки Security (1102) и процессы из Temp замечают только при целенаправленном поиске.

На операторе такие задачи обычно в SIEM; для одной рабочей станции нужны те же шаги: сбор, нормализация, правила [1].

Промышленные SIEM Splunk, IBM QRadar, Micro Focus ArcSight справляются с этими задачами успешно, однако стоят миллионы тенге лицензий, требуют круглосуточного SOC и кластера серверов. Для учебной задачи и небольшого стенда такое решение избыточно. Поэтому в качестве объекта разработки выбран упрощённый аналог – mini SIEM – на одну рабочую станцию: Python, SQLite, десктопное приложение на PySide6, правила в YAML. Идея состоит не в повторении промышленного продукта, а в демонстрации того, что вся механика SIEM укладывается в понятный код, который можно прочитать, отладить и расширить.

Тема актуальна по нескольким причинам. Атаки на учётные записи через RDP и SMB не теряют популярности, особенно после массового перехода на удалённую работу. PowerShell с EncodedCommand и техники Living-off-the-Land обходят антивирусы, потому что используют штатные утилиты ОС. Очистка журнала событий EventID 1102 часто сопровождается закреплением атакующего на хосте и уничтожением следов. Все эти признаки видны в стандартных журналах Windows – нужно только уметь их читать и сопоставлять во времени, чем и должна заниматься SIEM.

Цель работы — mini SIEM для одной Windows: сбор Security/System/Application/Sysmon, SQLite, правила YAML (в т.ч. корреляции), GUI с поиском и экспортом HTML/JSON.

Для достижения цели поставлены задачи:

- проанализировать предметную область мониторинга и ИБ в телекоммуникационной среде и на конечных точках Windows;

- выполнить сравнительный анализ существующих SIEM-решений и обосновать выбор упрощённой архитектуры для дипломного проекта;

- сформировать техническое задание на разработку mini SIEM с критериями приёмки;

- выбрать технологии и средства разработки Python, SQLite, PySide6, YAML, pywin32, PyInstaller;

- спроектировать архитектуру, модель данных и подсистемы сбора, детектирования и представления;

- реализовать программный комплекс и документацию по развёртыванию;

- провести тестирование на учебном стенде и подтвердить срабатывание правил детектирования.

Объект исследования – процессы сбора и первичного анализа событий безопасности на рабочей станции Windows.

Предмет – программные методы реализации конвейера обработки этих событий: чтение журналов через Win32 и Evt API, нормализация полей, хранение в SQLite, декларативные правила в YAML и поддержка работы аналитика.

Методы: NIST SP 800–92 [2], документация Microsoft по аудиту [3], обзор SIEM, модульная архитектура, Python, ручные тесты с EVTX.

Особенность задачи в том, что внутри одной рабочей станции нужно повторить логику, которую крупные SOC обычно строят на распределённой инфраструктуре. Это заставляет аккуратно проектировать каждый этап конвейера: где-то экономить ресурсы, где-то жертвовать масштабом ради простоты сопровождения и наглядности на защите.

За основу при выборе подходов приняты рекомендации NIST SP 800–92 по управлению журналами [2], документация Microsoft по аудиту Security [3] и публикации Microsoft Defender по типовым атакам на Windows. Из практики «ТрансТелеКом» учтён принцип единого часового пояса UTC для всех логов и обязательная фиксация действий аналитика – без этого сопоставление событий и закрытие инцидентов невозможно.

1 Предпроектные исследования

1.1 Анализ предметной области

SIEM на одном ПК сталкивает эксплуатацию (серверы, метрики) и ИБ (атаки, расследования).

У оператора это разные команды; на Windows всё сходится в Event Log [4].

Сама сеть оператора устроена многоуровнево: магистраль, агрегация, доступ, сервисные платформы AAA, биллинг, личный кабинет, почта [5]. Каждый уровень даёт собственные источники событий: NetFlow от маршрутизаторов, syslog с коммутаторов, RADIUS-логи аутентификации [6]. Разрабатываемый mini SIEM на этом фоне в проекте — лишь часть картины оператора, закрывающая endpoint и показывающую принципы чтения событий Windows. Подключение таких источников, как syslog или NetFlow, в перспективе сопоставимо по сложности с добавлением ещё одного collector.

Угрозы для Windows endpoint: доступность (DDoS) здесь вторична; важнее целостность (подмена бинарников, очистка журнала) и конфиденциальность (кража учёток, LSASS, утечка файлов) [7]. В журналах Security и Sysmon наблюдается большая часть признаков по второй и третьей группам, что и составляет основной фокус mini SIEM [8].

Мониторинг в операторской среде выполняет одновременно операционную, аналитическую и косвенно-безопасностную функции. Операционно важно раннее обнаружение отказов и деградаций; аналитически — выявление узких мест и планирование модернизации; для ИБ — обнаружение паттернов, требующих углублённого разбора. Типовой жизненный цикл инцидента эксплуатации включает регистрацию, классификацию, диагностику, локализацию, восстановление и постанализ. Для ИБ-критичных сценариев добавляется требование непрерывности и достоверности учёта кто и когда выполнял действия, синхронизации времени NTP, единая шкала UTC и полноты журналов на границах и в подсистемах аутентификации [9].

PON, Ethernet и Wi-Fi дают разные метрики (оптика, FCS, шум эфира).

Для ИБ на абонентском контуре важнее CPE, заводские пароли веб-морды и дыры в прошивках.

Операторы закрывают это политиками и сегментацией; СИБ должен это учитывать [5].

Полноценный SIEM у оператора тянет syslog, NetFlow, RADIUS, DHCP, прокси, DNS, AD [10].

В филиале до всех логов часто нет доступа — политика и коммерческая тайна.

В дипломе взяты только журналы Windows: Security, System, Application и при возможности Sysmon [3].

Так обрабатываются нормализация, поиск, пороги, корреляция и отчёты без продакшен-логов.

Корреляция – ключевая особенность SIEM по сравнению с обычным просмотром журнала. При фильтрации в Event Viewer по EventID 4625 формируется список неудачных входов. Корреляция дополняет такую выборку возможностью задать условие «подряд 5 событий за 5 минут с одного IP» и автоматически создавать алерт. К этому может быть присоединена цепочка: после серии 4625 регистрируется успешный 4624 что указывает на подбор пароля, а вслед за ним – запуск powershell.exe из Temp. В результате формируется описание реальной атаки, а не три разрозненных события.

На практике в ТТК цепочку «много 4625 → один 4624» смотрели вручную; в mini SIEM это правило WIN-CORR-BRUTE-LOGIN-PROC.

На рабочей станции Windows выделены типовые сценарии, подлежащие детектированию: перебор пароля по RDP и локальному входу, запуск PowerShell с EncodedCommand. Все эти сценарии описываются через измеримые признаки: счётчик событий за окно, конкретный EventID, подстрока в командной строке [11].

Отдельного внимания требует вопрос времени. Без точной синхронизации часов корреляция нарушается: события из разных источников попадают в окно случайным образом. В Windows этот вопрос решается NTP-синхронизацией с домен-контроллером или time.windows.com, а в самой записи сохраняется UTC. В mini SIEM значение ts_utc хранится в формате ISO 8601 и пересчитывается в выбранный пользователем часовой пояс только в GUI – что исключает путаницу при сопоставлении с внешними логами.

Наконец, предметная область включает организационные меры: политики паролей, разграничение ролей, обучение пользователей, регламенты эскалации. SIEM не заменяет эти меры, но гарантирует измеримость: сколько попыток входа, кто инициировал изменение, какие процессы запускались после компрометации учётной записи. В отчёте по практике это связывается с наблюдаемой в ИТ-отделе дисциплиной работы с тикетами и внутренними инструкциями.

ИБ оператора — техника, процессы и люди: сеть, платформы, данные абонентов [12].

В филиале много действий идёт с рабочих Windows — след остаётся в Event Log.

Требования СМИБ — ISO/IEC 27001:2022 и ГОСТ Р ИСО/МЭК 27001–2021 [13], [14].

Цикл в SOC: запись → доставка → нормализация → индекс → правило → алерт → triage → расследование → закрытие.

Mini SIEM сжимает это до одной машины; HTML/JSON-отчёт к разбору приложить можно.

Классификация угроз для конечной точки Windows включает перебор учётных данных локальный вход, RDP, SMB, использование легитимных утилит PowerShell, WMI, PsExec, закрепление через автозагрузку и задачи планировщика, попытки отключения защиты и очистки журналов. События Security 4624 и 4625 фиксируют успешные и неудачные попытки входа; событие

1102 сигнализирует об очистке журнала Security – критический индикатор возможного сокрытия следов [3]. Расширение Sysmon дополняет картину детализацией создания процессов и командных строк [15].

В дипломе это сводится к моделированию конвейера событийной аналитики: от первичного сигнала до алерта и действий аналитика. Это соответствует профессиональной логике SOC/SIEM и укладывается в объём индивидуального диплома [16].

На стажировке в ИТ-отделе пароли и эскалация без журналов плохо проверяются постфактум. Mini SIEM даёт цифры: сколько входов, кто менял настройки, что запускалось после входа — это удобно при разборе тикетов.

В филиале NOC смотрит линии и отказы; подозрительный вход на ПК сотрудника часто всплывает только по жалобе. В дипломе поэтому фокус на endpoint, а не на NetFlow.

Для одной рабочей станции «доступ» в ИБ — это RDP, VPN и офисный Wi-Fi: через них чаще идёт перебор пароля, а не через магистраль PON.

1.2 Анализ существующих разработок

Рынок SIEM сформировался из потребности крупных организаций в единой точке сбора событий безопасности, корреляции и управления инцидентами с учётом требований регуляторов. Ниже приведена сравнительная таблица трёх широко известных коммерческих платформ; оценки обобщённые и показывают типичные компромиссы при выборе, а не рекламные характеристики конкретной редакции.

Таблица 1.1

Сравнительная характеристика коммерческих SIEM обобщение

Критерий	Splunk + Enterprise Security	IBM QRadar	Micro Focus ArcSight OpenText
Позиционирование	Универсальная платформа машинных данных, модуль ES для ИБ	Корпоративный SOC, отчётность, интеграции	Крупные внедрения, сильный акцент на корреляции
Сильные стороны	Гибкий поиск, экосистема приложений, скорость итераций аналитики	Зрелые отчёты, развитые интеграции	Глубина корреляционной модели, коннекторы
Типовые ограничения	Совокупная стоимость владения, требования к экспертизе	Сложность внедрения и обновлений	Высокий порог сопровождения, TCO
Масштабирование	Высокое при правильной архитектуре кластера	Высокое	Высокое
Переносимые идеи в mini SIEM	Индексация и запросы к событиям	Правила и «офенсы», кейсы	Цепочки корреляций

Таблица 1.2

Обобщённые преимущества и ограничения класса enterprise SIEM

Аспект	Преимущества	Ограничения
Функциональность	Корреляции, кейсы, threat intelligence, отчёты комплаенса	Избыточность для малого контура и учебного стенда
Эксплуатация	Зрелые практики SOC	Необходимость выделенных ролей и регламентов 24/7
Экономика	Снижение рисков при зрелом процессе	Лицензирование по объёму событий EPS, стоимость хранения
Гибкость	Много готовых коннекторов	Зависимость от вендорской дорожной карты и поддержки

Помимо коммерческих решений широко используются открытые и гибридные стеки. Wazuh развитие идей OSSEC популярен в связке с Elastic Stack: агенты на конечных точках, сервер правил, интеграция с Elasticsearch для поиска и Kibana для визуализации [17]. Elastic Stack как основа лог-платформы даёт мощный полнотекстовый поиск и агрегации, но требует администрирования кластера и дисциплины в управлении индексами и жизненным циклом данных [18]. Apache Kafka в сочетании с column-oriented хранилищами например, ClickHouse применяется в высоконагруженных конвейерах потоковой обработки событий; для дипломного проекта одного исполнителя такая архитектура обычно избыточна, но полезна как ориентир при описании масштабирования [16].

Уровни зрелости SOC в литературе описываются от реагирования по входящим алертам до проактивного threat hunting. В филиальной модели телеком-оператора полноценный SOC зачастую централизован на уровне холдинга, а локальная ИТ-служба выполняет функции первой линии: приём симптомов, первичная классификация, эскалация. SIEM в этой модели – инструмент аналитиков второй-третьей линии и центральной ИБ-службы. Учебный mini SIEM делает возможным смоделировать элементы второй линии на масштабе одной рабочей станции: правила, окна времени, корреляции и работа с алертами.

Экономика владения TCO коммерческого SIEM включает лицензии на события в секунду, горячее и архивное хранение, персонал, обучение, интеграции с ITSM и SOAR. Для вузовского проекта и малых организаций разумно использовать упрощённый стек с локальной СУБД и десктопным интерфейсом, сохраняя при этом научно-практическую ценность: демонстрацию того, что ядро SIEM – это качество конвейера нормализации, хранения и детектирования, а не только бренд платформы.

Таблица 1.3

Сопоставление подходов: enterprise SIEM и дипломный mini SIEM

Параметр	Enterprise SIEM	Mini SIEM
Источники	Множество сетевых и серверных систем	Windows Event Log, опционально Sysmon, импорт EVTX
Хранение	Кластер, политики tiering	SQLite, WAL, локальный файл в профиле пользователя
Правила	Большие библиотеки, команды аналитиков	YAML, параметризуемые детекторы, корреляции ограниченного набора
UI	Веб-порталы SOC	Десктоп PySide6, отчёты HTML/JSON
Цель внедрения	Производственный SOC	Учебный стенд, пилот, демонстрация компетенций

Среди облачных и гибридных предложений встречаются SIEM как сервис обобщённо cloud SIEM с оплатой по объёму и встроенными коннекторами к Office 365, облачным провайдерам и endpoint detection and response. Такие решения снижают операционную нагрузку на заказчика, но создают зависимость от вендора, вопросы суверенитета данных и стоимости при росте объёма логов. Для дипломного исследования облачный SIEM может рассматриваться как ориентир по функциональности: кейсы, playbook, интеграция с ITSM, но не как обязательная реализация.

Аналитика ложных срабатываний в зрелых SIEM приводит к появлению дисциплин тюнинга правил, белых списков, машинного обучения и поведенческих моделей. В mini SIEM объём данных меньше, но принцип сохраняется: пороги burst по неудачным входам должны подбираться с учётом легитимных сценариев скрипты, устаревшие клиенты, общие учётные записи. Поэтому в ТЗ закреплена параметризация threshold и window_minutes в YAML, это тот же тюнинг, что в промышленных SIEM.

Порог burst (5 за 5 мин) подбирал на EVTX: при 3 — тихо, при 10 — шум; для защиты оставил 5.

Интеграция с MITRE ATT&CK в коммерческих продуктах часто сделана в виде тегов тактик и техник, панелей покрытия и рекомендаций по реагированию. В дипломном проекте ссылки на тактики например, перебор паролей, удалённые сервисы RDP, сокрытие следов, использование PowerShell могут включаться в метаданные правил; это повышает научную культуру отчёта и облегчает сопоставление детекторов с международной терминологией [11].

SOAR автоматизирует блокировки и тикеты поверх SIEM.

В mini SIEM пока только журнал actions и dry-run по умолчанию.

Так же в SOC: сначала тюнят правила, потом включают авто-реакции.

Итог обзора: для диплома по СИБ подходит локальный стек с прозрачным кодом, SQLite и YAML-правилами [19]. Масштабирование — отдельный collector и очередь, без смены модели события.

Splunk/QRadar на кафедре в пилот не ставили — сравнение по документации и открытым кейсам, не по продакшену.

Открытые решения на базе Wazuh и Elastic Stack дают гибкость и отсутствие лицензионных платежей за ядро, но требуют администрирования кластера индексов, настройки агентов и дисциплины управления жизненным циклом данных. Apache Kafka в связке с column-oriented хранилищами применяется в высоконагруженных конвейерах потоковой обработки; для индивидуального дипломного проекта такая архитектура избыточна, однако служит ориентиром при описании перспектив масштабирования.

Комиссии важнее цепочка collector → rules → alerts на живом EVTX, чем название вендора — это покажу на защите.

Облачные SIEM в тексте только для сравнения: в ТЗ — офлайн-машина, интернет не обязателен.

Итог обзора: для защиты хватит прозрачного стека Python + SQLite + YAML; при росте нагрузки collector и БД вынесут на сервер без смены модели события.

Wazuh с Elastic тянет кластер индексов — на одном ПК тяжело; в mini SIEM поиск по событиям закрывают SQLite и фильтры GUI.

Стенд имитирует уровень «аналитик разбирает алерт»: статусы new/in_progress/closed и комментарий — без круглосуточной смены SOC.

Mini SIEM стоит как один ПК и время разработки; плата за EPS, как у QRadar, в проекте не считалась — это осознанное упрощение.

1.3 Постановка задачи

Что нужно сделать: разработать программный комплекс mini SIEM для мониторинга и анализа событий безопасности на рабочей станции под Microsoft Windows. Не enterprise-платформа, не агентская сеть, не облачный сервис — именно компактное настольное приложение, которое можно развернуть на одной машине и продемонстрировать на защите [16].

Источник постановки задачи. Тема выросла из практики в «ТрансТелеКом» Астана, где был получен опыт работы локальной ИТ-службы с событиями Windows на рабочих станциях. Полноценный SOC у оператора централизован на уровне холдинга, а на местах часто используются доступные средства: Event Viewer и базовые скрипты. Именно для таких сценариев требуется компактный SIEM, прототип которого выбран в качестве темы дипломного проекта.

Цель — собрать прототип, который умеет: читать журналы Windows Security, System, Application и Sysmon, сохранять события в локальную SQLite, искать и фильтровать их, применять правила детектирования в том числе корреляционные «перебор → вход → процесс», показывать оператору алерты, формировать HTML- и JSON-отчёты по инциденту и регистрировать действия аналитика.

Назначение: использование в учебно-исследовательских целях, локальный разбор инцидентов с импортом файлов EVTX, демонстрация на защите диплома.

Предмет разработки: программные методы и средства реализации конвейера «источник – сбор – нормализация – хранение – детектирование – представление – отчёт».

Источники: Security, System, Application; Sysmon — если установлен и есть права.

«Длинные» каналы — Evt API; классические — pywin32 с маской EventID.

Импорт EVTX — для офлайн-разбора на защите [3].

Требования к хранению. Использовать СУБД SQLite с включённым режимом WAL, внешними ключами, где применимо, индексами по полю времени UTC и составными индексами для ускорения типовых запросов правил и интерфейса. Логическая схема должна включать, как минимум: таблицу событий время UTC, канал, поставщик, идентификатор события, уровень, компьютер, номер записи, текст сообщения, JSON с деталями, поля для разобранных параметров входа для событий 4624 и 4625; таблицу алертов идентификатор правила, серьёзность, заголовок, детали, статус, комментарий; таблицу действий; таблицы состояния и параметров правил; таблицу настроек.

Функциональные требования:

- FR-1 — сбор из каналов по таймеру;
- FR-2 — поиск/фильтр по времени, каналу, EventID, тексту;
- FR-3 — правила в YAML, вкл/выкл, пороги и окна;
- FR-4 — детекторы: burst 4625, RDP type 10, 1102, EncodedCommand, цепочка «ошибки → вход → процесс» [11];
- FR-5–8 — алерты, отчёты HTML/JSON, dry-run, PyInstaller — как в ТЗ.

Нефункциональные требования NFR. NFR-1: производительность – опрос журналов пакетами ограниченного размера; тяжёлые операции не должны блокировать интерфейс использование рабочих потоков. NFR-2: надёжность – корректная фиксация транзакций, административные операции обслуживания WAL checkpoint, vacuum по запросу, политики удаления устаревших данных. NFR-3: сопровождаемость – README, модульная структура db, collector_win, rules, app, evt_x_import, report, sysmon. NFR-4: безопасность – отсутствие скрытых опасных действий; контроль путей к файлу базы в профиле пользователя; осознанная модель прав доступа Windows для чтения журнала безопасности.

Состав поставки: исходный код; файл правил; инструкция по установке Python и зависимостей; команда запуска python -m minisiem; опционально готовый miniSIEM.exe.

Критерии приёмки: сбор с заявленных каналов без аварийного завершения при нормальной нагрузке ПК; сохранение событий в SQLite и отображение в GUI; демонстрация срабатывания не менее трёх правил на тестовых данных или импорте EVTX; наличие алертов в таблице и возможность фильтрации; документация, достаточная для развёртывания на машине разработчика.

Первый запуск без фонового потока подвисал GUI — collector перенесён в QThread.

Таблица 1.4

Матрица соответствия требований и модулей проекта mini SIEM

Требование	Реализация в проекте
Сбор классических журналов	collector_win.py, Win32 Event Log API
Сбор Sysmon	collector_win.py, Evt API
Импорт EVTX	evtx_import.py
Миграции и запросы БД	db.py
Правила и детекторы	rules.py, rules/default.yml
Интерфейс	app.py, PySide6
Отчёты	report.py
Справка по Sysmon	sysmon.py

Ожидаемый результат: работоспособный программный комплекс mini SIEM, подтверждающий на защите диплома получение целей и критериев приёмки, с возможностью дальнейшего развития экспорт в syslog, интеграция с внешним коллектором, threat intelligence.

Роли на стенде: администратор (установка, пути, хранение, опасные функции) и аналитик (события, алерты, комментарии, отчёты).

На одной Windows разграничение — учётки и права на файл БД.

В ТЗ: без паролей и продакшен-учёток в репозитории.

Границы и исключения. В объём работ не входит замена корпоративного SIEM оператора; не входит сертификация средств криптографической защиты; не входит интеграция с биллингом и CRM; не входит поддержка не-Windows платформ. Эти ограничения снижают риск неконтролируемого расширения диплома и делают критерии приёмки проверяемыми.

Риски проекта и меры. Риск «шумных» правил: митируется параметрами порогов и тестовыми EVTX. Риск блокировки UI: митируется потоками. Риск компрометации базы с событиями: митируется размещением в профиле пользователя, политиками ОС и рекомендациями по шифрованию диска. Риск небезопасных реакций на алерты: митируется dry-run по умолчанию.

Требования к документации для пользователя. README должен содержать требования к версии Python, команды создания venv и установки зависимостей, команду запуска, краткое описание включения Sysmon, инструкцию по сборке exe. Для защиты диплома дополнительно готовится демонстрационный сценарий.

Требования к тестированию. Необходимо провести ручные тесты: холодный старт на пустой базе; импорт EVTX; проверка срабатывания правил на синтетических данных; длительный прогон несколько часов на рабочей станции с обычной нагрузкой для выявления утечек памяти или роста файла БД; проверка корректности времени UTC при смене часового пояса системы.

Требования к сопровождению кода. Структура пакета minisiem должна оставаться читаемой; миграции схемы БД – идемпотентными скриптами; добавление новых полей – через мягкие ALTER TABLE с обработкой уже существующих колонок, чтобы не ломать установки преподавателя или рецензента при обновлении версии проекта.

Техническое задание на разработку mini SIEM формулирует функциональные требования FR-01–FR-08: сбор из каналов Security, System, Application и Sysmon; сохранение в SQLite; поиск и фильтрация; применение правил; формирование алертов; управление статусами; экспорт отчётов; импорт EVTХ. Нефункциональные требования NFR-01–NFR-05 касаются отзывчивости интерфейса при фоновом сборе, переносимости поставки, безопасного режима реакций, документирования развёртывания и воспроизводимости срабатываний правил на тестовых данных.

Критерии приёмки включают: подтверждённый сбор событий из заявленных каналов; наличие записей в базе данных; отображение во вкладке Events; срабатывание не менее трёх правил детектирования на подготовленном наборе; фиксация алертов с уровнями серьёзности; возможность экспорта HTML-отчёта; успешная сборка исполняемого файла miniSIEM.exe.

2 Выбор технологий и средств разработки

2.1 Язык программирования и архитектура системы

Основной язык — Python 3.11 x64.

На нём быстро собрать прототип: текст, время, потоки, `pywin32` к журналам.

Сбор логов — в основном ожидание I/O; при пакетном чтении и записи в SQLite узким местом Python не был [20].

Для mini SIEM выбран модульный стиль с разделением этапов «сбор → хранение → анализ → представление». Модульность иллюстрирует промышленную архитектуру SIEM, где сборщики и хранилища часто независимы и масштабируются различным образом. Декомпозиция сводится к отдельным модулям базы данных `db.py`, правил `rules.py` и YAML-файл, чтения журналов Windows `collector_win.py`, импорта архивов `evtx_import.py`, отчётности `report.py` и графического приложения `app.py`. В операторской среде критичные логи находятся в зоне ограниченного доступа; для дипломного проекта целесообразен локальный контур, воспроизводимый на стенде без доступа к production.

Выбор Python 3.11 связан со зрелой экосистемой библиотек для Windows `pywin32`, GUI `PySide6`, конфигурации `PyYAML` и упаковки `PyInstaller`. Интерпретируемая природа языка ускоряет итерации при разработке правил и интерфейса; для доминирующего режима ожидания ввода–вывода при чтении журналов производительность интерпретатора не становится критическим ограничением при пакетном чтении и индексированном хранении в SQLite.

Модульная архитектура наглядно отражает промышленное разделение ответственности: `collector_win` не содержит логики правил; `rules` не обращается к Win32 API напрямую; `db` изолирует SQL-схему; `app` координирует потоки и представление. Такое разделение упрощает тестирование отдельных компонентов, замену хранилища в перспективе и демонстрацию структуры проекта на защите диплома.

Локальный контур разработки и эксплуатации выбран сознательно: отсутствует необходимость доступа к production-логам оператора; стенд воспроизводится на виртуальной машине или ноутбуке с включённым аудитом Windows. Журналы Security, System, Application и Sysmon покрывают типовые сценарии компрометации рабочей станции и достаточны для демонстрации корреляционных правил.

Среда разработки включает виртуальное окружение Python, `requirements.txt` и Git для воспроизводимости между машинами студента и научного руководителя. Сборка выполняется под Windows 10/11 x64; `pywin32` и `PyInstaller` зависят от платформы и разрядности.

Контроль качества берёт на себя единой структурой пакета `minisiem`, изоляцией зависимостей в `venv` и проверками на «чистой» машине: установка `requirements.txt`, запуск `python -m minisiem`, импорт EVTX.

Python PSF license, PySide6 LGPL, SQLite public domain – юридическая совместимость стека для учебного использования без закупки лицензий коммерческих SIEM.

2.2 Сбор и нормализация журналов Windows

Security/System/Application читаются через Win32.

Канал Sysmon — через Evt API: legacy OpenEventLog «длинные» имена не тянет [21].

EventID режут маской 0xFFFF — иначе правила по 4625/4624 сбиваются [3].

При отладке collector без маски 0xFFFF для legacy EventID правило 4625 молчало — в коде это исправлено.

Из Python к API подключается pywin32 [21].

Качество детектирования зависит от стабильной и единообразной интерпретации полей событий при разных версиях Windows и разных источниках классический журнал и Sysmon. В mini SIEM применён подход raw-normalized: события приводятся к общему набору полей время, канал, provider, EventID, level, computer, record_id, message и дополняются JSON-полем с исходными деталями. Подход сохраняет доказательную базу и гарантирует универсальные SQL-запросы для фильтрации и правил.

Для событий входа Security 4624/4625 в аналитике значимы учётная запись, тип входа в том числе удалённые сценарии и IP-адрес источника. В Windows часть данных находится в массиве строковых вставок с форматом, зависящим от локали и версии. В проекте предусмотрено хранение сырого массива вставок JSON и отдельных разобранных полей parsed_account, parsed_ip, parsed_logon_type для ускорения запросов. Sysmon рассматривается как опциональный, но рекомендуемый источник для детектов PowerShell с EncodedCommand [15].

Доступ к классическим журналам сделан через функции Win32 API; для каналов Analytical и Operational с символом «/» в имени применяется Evt API, поскольку универсальное чтение через OpenEventLog для таких каналов не гарантируется. Единый модуль collector_win инкапсулирует обе ветви, снижая риск ошибок конфигурации при добавлении новых источников.

Нормализация EventID с маской 0xFFFF устраняет расхождения между Win32 и Evt-представлением идентификатора события. Поле data_json сохраняет исходные детали в формате JSON для доказательной базы; колонки parsed_account, parsed_ip и parsed_logon_type ускоряют агрегации для событий входа 4624/4625 без повторного разбора текстовых вставок при каждом запросе.

Импорт EVTХ через evtx_import.py дополняет онлайн-сбор возможностью ретроспективного анализа: архив журнала загружается пакетами, проходит ту же нормализацию и участвует в детектировании наравне с событиями реального времени. Это критично для демонстрации на защите без генерации атаки на живой системе.

Кодировки и локализация сообщений Windows влияют на детекторы по подстрокам командной строки. Для учебного стенда допустима фиксация одной локали; для промышленного развития потребовались бы нормализованные поля или multilingual-паттерны.

Подход raw-normalized сохраняет StringInserts в data_json и одновременно заполняет typed-колонки для SQL-агрегаций – компромисс между доказательной базой и производительностью.

Модель угроз на уровне ОС включает перебор RDP/SMB, PowerShell с EncodedCommand, очистку журнала 1102, запуск из temp-каталогов. Встроенный аудит Windows выполняет базовую картину; Sysmon расширяет детализацию хэшами, родительским процессом и командной строкой.

Разделение Win32 и Evt API в collector_win.py документировано в комментариях модуля и пояснительной записке для предотвращения ошибочного подключения Sysmon через OpenEventLog.

2.3 Хранение данных и правила детектирования

Хранилище — SQLite в одном файле, без отдельного сервера на стенде. SQL хватает для агрегаций и окон в правилах.

WAL снижает блокировки, когда сборщик пишет, а GUI читает.

Файл в %LOCALAPPDATA%\miniSIEM; таблицы events, alerts, actions повторяют практику SIEM [22].

Формат YAML выбран для человекочитаемости правил и версионирования в системе контроля версий. Разбор YAML-файлов выполняется средствами библиотеки PyYAML. Правило содержит идентификатор, заголовок, описание, уровень серьёзности, имя детектора и блок параметров. Пороги чувствительности вынесены в параметры threshold и window_minutes. Для демонстрации на защите предусмотрены синтетические сценарии и импорт EVTХ с заранее подготовленным набором событий. Для локального SQLite предусмотрены операции обслуживания: удаление записей старше заданного срока, checkpoint WAL и vacuum [23].

Файл БД в %LOCALAPPDATA%\miniSIEM: перенос стенда = скопировать один .sqlite3, без установки сервера PostgreSQL.

YAML-файл правил отделяет политику детектирования от кода. Каждое правило содержит идентификатор, заголовок, severity, имя детектора и параметры threshold, window_minutes. Таблицы rules_state и rule_params в SQLite хранят состояние включения и пользовательские пороги, изменённые через GUI.

Политика ретенции сделана функциями prune_all_older_than, checkpoint_wal и vacuum. Без периодической очистки файл базы на активной рабочей станции будет неограниченно расти; параметр срока хранения задаётся во вкладке Settings и применяется административной процедурой обслуживания.

Индексы idx_events_ts, idx_events_channel_ts и idx_events_eventid_ts спроектированы под профиль детекторов: скользящие окна по времени,

фильтрация по каналу Security/Sysmon и конкретным EventID 4625, 4624, 1102. Обоснование – снижение стоимости каждого цикла опроса правил при росте таблицы events.

Таблицы events, alerts и actions демонстрируют процесс «обнаружение → разбор → решение». Таблица actions сохраняет тип действия, связь с алертом, результат и детали – в том числе в режиме dry-run.

Пример фрагмента правила YAML для иллюстрации: id WIN-SEC-4625-BURST, detector failed_logon_burst, params threshold 5 и window_minutes 5. Такая декларация отделяет политику от кода детектора в rules.py.

Конфиденциальность данных в SQLite: учётные имена, IP-адреса. Рекомендуется шифрование диска BitLocker и отдельные учётные записи ОС для учебного стенда.

2.4 Интерфейс, упаковка и эксплуатация

Для десктопной Windows-среды выбран фреймворк Qt через биндинги PySide6: нативный внешний вид, табличные представления для больших объёмов событий, фильтрация по времени и каналу, поиск по EventID, просмотр алертов и смена статуса обработки. Веб-интерфейс увеличил бы объём фронтенд-разработки и усложнил демонстрацию без поднятого сервера [24].

PyInstaller помогает собрать исполняемый файл miniSIEM.exe для переноса на машину без установленного Python. Стек разработки включает pywin32, PyYAML, sqlite3, PySide6; виртуальное окружение Python, requirements.txt и Git для воспроизводимости. Сбор событий и тяжёлые запросы к БД выполняются вне главного потока UI в отдельных потоках Qt [25].

Чтение журнала Security требует привилегий; реакции на алерты по умолчанию выполняются в режиме симуляции. Файл SQLite может содержать учётные имена и IP-адреса; доступ ограничивается политиками ОС. Демонстрации проводятся на тестовом стенде без данных реальных клиентов.

PySide6 берёт на себя нативный десктопный интерфейс с табличными моделями QAbstractTableModel, привязанными к SQL-запросам с лимитом строк. Фоновый сбор и импорт EVTХ выполняются в отдельных потоках Qt QThread, QObject-воркеры, что предотвращает блокировку главного окна при длительных операциях.

PyInstaller упаковывает интерпретатор, зависимости и ресурсы в miniSIEM.exe согласно спецификации packaging/minisiem.spec. Поставка в виде одного файла упрощает демонстрацию на машине без установленного Python; ограничением остаётся необходимость тестирования сборки на целевой ОС и возможные срабатывания антивируса на неподписанный исполняемый файл.

Режим dry-run для реакций на алерты включён по умолчанию: действия записываются в таблице actions как симуляция без реального воздействия на систему. Такой подход согласуется с принципами безопасной учебной

эксплуатации и снижает риск необдуманных операций при разборе тестовых инцидентов.

Модуль `theme.py` гарантирует единообразное оформление таблиц; `autostart.py` управляет автозапуском `exe` через реестр Windows. Модуль `time_display` преобразует UTC в выбранный часовой пояс отображения без изменения хранимых значений `ts_utc`.

Управление доступом к артефактам: HTML–отчёты и файл SQLite следует хранить в защищённом каталоге; в `production` – политики DLP и шифрование носителей.

Альтернатива веб–интерфейсу FastAPI + React отклонена из–за необходимости поднятого сервера и увеличения поверхности атаки HTTP–порт. Десктопное приложение не требует сетевых служб по умолчанию.

Методика оценки производительности: скорость импорта EVTХ, время обновления таблицы за 24 ч, время HTML–отчёта, рост файла базы. Пакет `batch_size=128` и интервал опроса 5–10с сглаживают пики.

2.5 Сравнение с альтернативами и итоговый стек технологий

Elasticsearch или OpenSearch обеспечили бы полнотекстовый поиск и горизонтальное масштабирование, но потребовали бы JVM–кластера и отдельной эксплуатации – за рамками простоты дипломного стенда. PostgreSQL с партиционированием – компромисс для одного сервера; для `single-user` приложения SQLite остаётся прямолинейным выбором. Дипломный проект намеренно использует минимально достаточную инфраструктуру для снижения трудозатрат и рисков демонстрации.

PySide6 распространяется под LGPL; SQLite – public domain; Python – PSF license. Стек не требует закупки коммерческих лицензий SIEM. Перспективы развития: экспорт в syslog/JSON Lines, PostgreSQL, брокер сообщений, внешние списки IOC, доставка алертов по e–mail или Telegram.

Таблица 2.1

Стек технологий mini SIEM итоговая

Компонент стека	Технология	Роль в системе
Язык	Python 3.11 x64	Логика, правила, интеграция
Доступ к ОС	pywin32, Evt API	Сбор журналов
БД	SQLite WAL	Хранение событий и алертов
Правила	YAML	Политика детектирования
GUI	PySide6 Qt 6	Визуализация и управление
Сборка	PyInstaller	Поставка <code>exe</code>
Импорт	EVTХ	Офлайн–расследования

Версии зависимостей записываются в requirements.txt; используется виртуальное окружение для исключения дрейфа библиотек. Сборка выполняется под Windows 10/11 x64; pywin32 и PyInstaller зависят от платформы. Сообщения журналов Windows могут быть локализованы; детекторы по подстрокам командной строки требуют учёта языка интерфейса на учебном стенде.

Сравнение с Elasticsearch/OpenSearch показывает: полнотекстовый поиск и горизонтальное масштабирование достигаются ценой JVM–кластера и операционной сложности. PostgreSQL с партиционированием – разумный компромисс при переходе к серверному хранилищу; для single–user десктопного приложения SQLite остаётся прямолинейным решением с возможностью последующей замены слоя db.py без изменения модели события [18].

Лицензионная чистота стека Python PSF, PySide6 LGPL, SQLite public domain исключает необходимость закупки коммерческих лицензий SIEM для учебного стенда. Перспективы развития включают экспорт в syslog/JSON Lines, централизованный коллектор, доставку алертов по e–mail или Telegram и расширение библиотеки правил с привязкой к тактикам MITRE ATT&CK.

Связь выбора технологий с практикой оператора: качество разбора инцидента зависит от полноты записей и единой шкалы времени. Таблица actions и комментарии к алертам воспроизводят культуру фиксации действий при разборе обращений в ИТ–отделе.

Сравнение с PostgreSQL: партиционирование по ts_utc, многопользовательский доступ, репликация – перспектива при росте от single–user к серверному хранилищу SOC.

3 Проектирование программного комплекса mini SIEM

3.1 Общая архитектура и конвейер обработки событий

Mini SIEM — десктоп на одной Windows без сервера сбора.

Модули разделены: сбор, нормализация, БД, правила, GUI — обмен словарями событий и SQL.

Тот же принцип «сбор → нормализация → хранение → детект → экран», что в IDS/SIEM [16].

Логическая архитектура комплекса mini SIEM показана на рисунке 3.1.

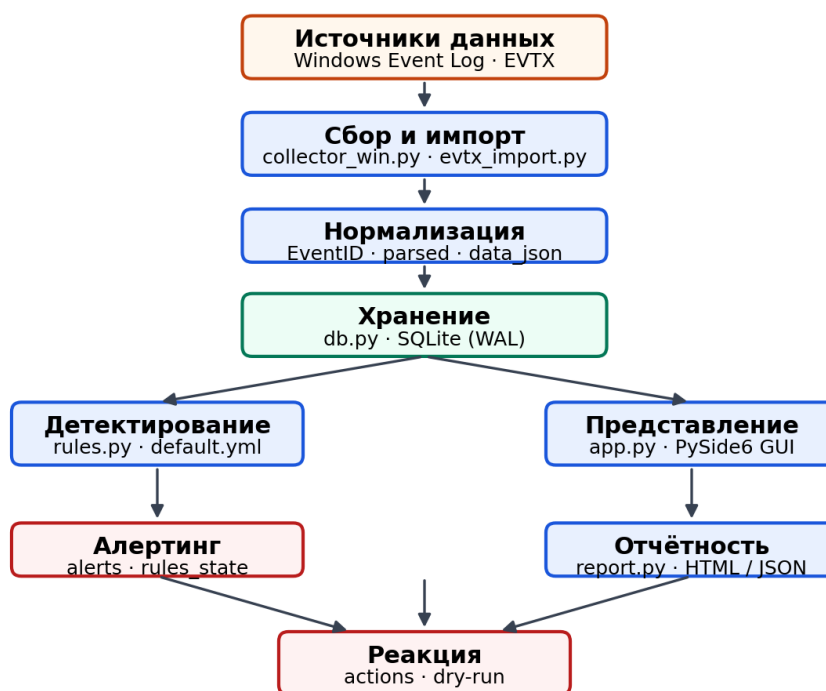


Рисунок 3.1 – Архитектура mini SIEM логические компоненты

Конвейер обработки включает следующие этапы:

- сбор Collection – периодический опрос каналов Windows Event Log и/или разовый импорт файлов EVT X;
- нормализация Normalization – приведение записи к единой схеме полей, извлечение вспомогательных атрибутов в т.ч. parsed_account, parsed_ip, parsed_logon_type для событий 4624/4625;
- хранение Storage – запись в SQLite, индексирование по времени и ключевым атрибутам;
- детектирование Detection – применение набора правил YAML к выборке событий за скользящее окно времени;
- алертинг Alerting – создание записей alerts с уровнем серьезности и дедупликацией повторных срабатываний;

- представление Presentation – отображение в GUI, поиск, панель статистики, экспорт HTML/JSON;

- реакция Response – журналирование предлагаемых или выполненных действий в таблице actions режим dry-run по умолчанию.

Проектирование ориентировано на расширяемость: добавление нового детектора сводится к реализации функции в модуле rules.py и описанию правила в YAML без изменения схемы БД; добавление источника – к расширению сборщика или импортёра. Такой подход отвечает требованиям практике промышленных SIEM, где политика детектирования отделена от кода платформы.

Выбор однопроцессной модели с фоновыми потоками Qt связан с требованием NFR-01 не блокировать интерфейс при опросе журналов и тяжёлых SQL-запросах. Сборщик и цикл правил выполняются вне главного потока GUI; обновление таблиц инициируется сигналами Qt по завершении операций.

Сбор Collection выполняется таймером QTimer с интервалом из settings. Нормализация Normalization приводит запись к единой схеме. Детектирование Detection запускается после вставки новых событий. Алертинг Alerting выполняет запись alerts с severity и dedup. Presentation – GUI и report; Response – actions с dry-run.

Модули развязаны: SQLite позже можно сменить на PostgreSQL, сохранив интерфейсы insert_event и search_events. Сборщик может быть вынесен в отдельный процесс или службу Windows при масштабировании.

При эксплуатации все компоненты размещаются на одной рабочей станции; сетевое взаимодействие с внешними коллекторами не предусмотрено, что снижает поверхность атаки самого средства мониторинга.

3.2 Проектирование хранилища данных

Хранилище сделано на базе SQLite 3 с режимом журналирования WAL Write-Ahead Logging. Файл базы размещается в каталоге данных пользователя %LOCALAPPDATA%\miniSIEM\minisiem.sqlite3, что выполняет требования модели разграничения доступа ОС и не требует прав записи в системные директории.

Связи простые: events → alerts → actions; пороги правил — в rules_state/rule_params, чтобы менять чувствительность без правки YAML на диске.

Таблица 3.1

Логическая схема основных таблиц

Таблица	Назначение	Ключевые поля
events	Нормализованные события	ts_utc, channel, event_id, message, data_json, parsed
alerts	Алерты правил	rule_id, severity, title, status, comment
actions	Журнал реакций	alert_id, action_type, result, details
rules_state	Вкл./выкл. правил	rule_id, enabled
rule_params	Параметры порогов	rule_id, key, value
settings	Настройки приложения	key, value

Индексы спроектированы под профиль запросов детекторов:

- idx_events_ts – выборки за скользящее окно;
- idx_events_channel_ts – фильтрация по каналу Security/Sysmon;
- idx_events_eventid_ts – фильтрация по EventID 4625, 4624, 1102 и др.

Поле data_json содержит сериализованную структуру с исходными вставками StringInserts и вспомогательными полями, что сохраняет доказательную базу для расследования. Отдельные столбцы parsed_account, parsed_ip, parsed_logon_type ускоряют группировку в правилах burst-by-IP и RDP-сценариях без полного разбора JSON на каждом запросе.

Для таблицы alerts предусмотрены статусы обработки new, in_progress, closed – в соответствии с реализацией GUI и поле comment для фиксации выводов аналитика. Механизм alert_exists_recently в слое БД снижает дублирование одинаковых алертов при частом опросе правил.

Политика хранения проектируется как административная операция: функции prune_older_than, checkpoint_wal, vacuum обеспечивают возможность ограничивать рост файла базы на длительно работающем стенде.

Схема данных включает таблицы events, alerts, actions, settings, rules_state, rule_params. Таблица events хранит нормализованные поля и data_json с исходными деталями; индексы idx_events_ts, idx_events_channel_ts и idx_events_eid_ts ускоряют временные выборки и фильтрацию по EventID.

Таблица alerts содержит rule_id, severity, title, status new/in_progress/closed, comment и связь с ключевыми событиями. Таблица actions протоколирует реакции аналитика или автоматические действия в режиме dry-run. Внешние ключи дают ссылочную целостность между алертами и действиями.

ER-диаграмма графический материал визуализирует связи сущностей и обосновывает выбор отдельных таблиц вместо монолитного хранения всех данных в одной структуре. Миграции в migrate создают схему идемпотентно при каждом запуске приложения.

Структура связей между сущностями базы данных приведена на рисунке 3.2.

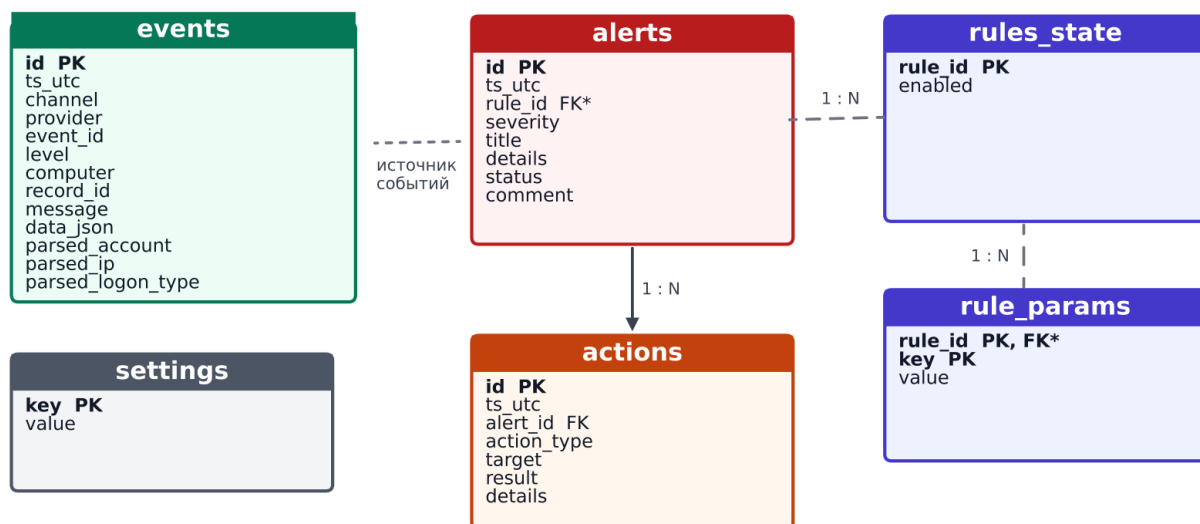


Рисунок 3.2 – ER–диаграмма базы данных mini SIEM

Таблица settings хранит ключевые параметры приложения: интервал опроса, срок хранения, смещение часового пояса, флаг dry-run. Миграции migrate выполняются идемпотентно при каждом запуске; добавление колонок parsed_* сделано через проверку _events_has_parsed_cols и ALTER TABLE.

Функция backfill_security_logon_parsed_fields обновляет до 5000 исторических записей 4624/4625 после миграции схемы. normalize_legacy_event_ids исправляет EventID в до 200 000 строк.

VACUUM выполняется после массового prune для reclaim дискового пространства. checkpoint_wal сливает WAL в основной файл и снижает размер wal/-shm артефактов.

Связь alerts и actions сделана внешним ключом alert_id. Статусы new, in_progress, closed поддерживают типовой workflow SOC: triage, расследование, закрытие с комментарием. Поле comment фиксирует выводы аналитика для последующего аудита.

3.3 Проектирование подсистемы сбора и нормализации

Подсистема сбора collector_win.py поддерживает два механизма чтения журналов Windows:

Legacy Win32 API OpenEventLog / ReadEventLog – для классических журналов Security, System, Application;

Windows Eventing API Evt\ – для каналов с косой чертой в имени, в частности Microsoft-Windows-Sysmon/Operational.

Sysmon на ноутбуке без админ-прав не читается — на защите покажу заранее импортированный EVTX с EncodedCommand.

Разделение связано с ограничениями платформы: универсальное чтение «длинных» каналов через legacy-API ненадёжно на ряде сборок pywin32 [21].

Опрос выполняется пакетами ограниченного размера `batch_size`, по умолчанию 128 записей с курсором по `RecordNumber` при чтении «назад» от новых к старым, уже прочитанные записи не дублируются.

Пакет 128 записей: при 512 UI лагал, при 64 опрос отставал — остановился на 128.

Нормализация включает:

- приведение EventID к младшим 16 битам для legacy-записей;
- формирование полей `channel`, `provider`, `level`, `computer`, `record_id`, `message`;
- для Security 4624/4625 – разбор учётной записи, IP-адреса источника и типа входа в т.ч. `logon type 10` для RDP;
- сохранение полного массива вставок в JSON.

Подсистема импорта EVTХ `evtx_import.py` предназначена для офлайн-расследований и демонстрации на защите без генерации «живого» трафика на рабочей машине комиссии. Импорт ограничивается параметром `limit` по умолчанию 2000 событий для защиты от переполнения памяти при больших архивах.

Последовательность вызовов при поступлении новой порции событий показана на рисунке 3.3.

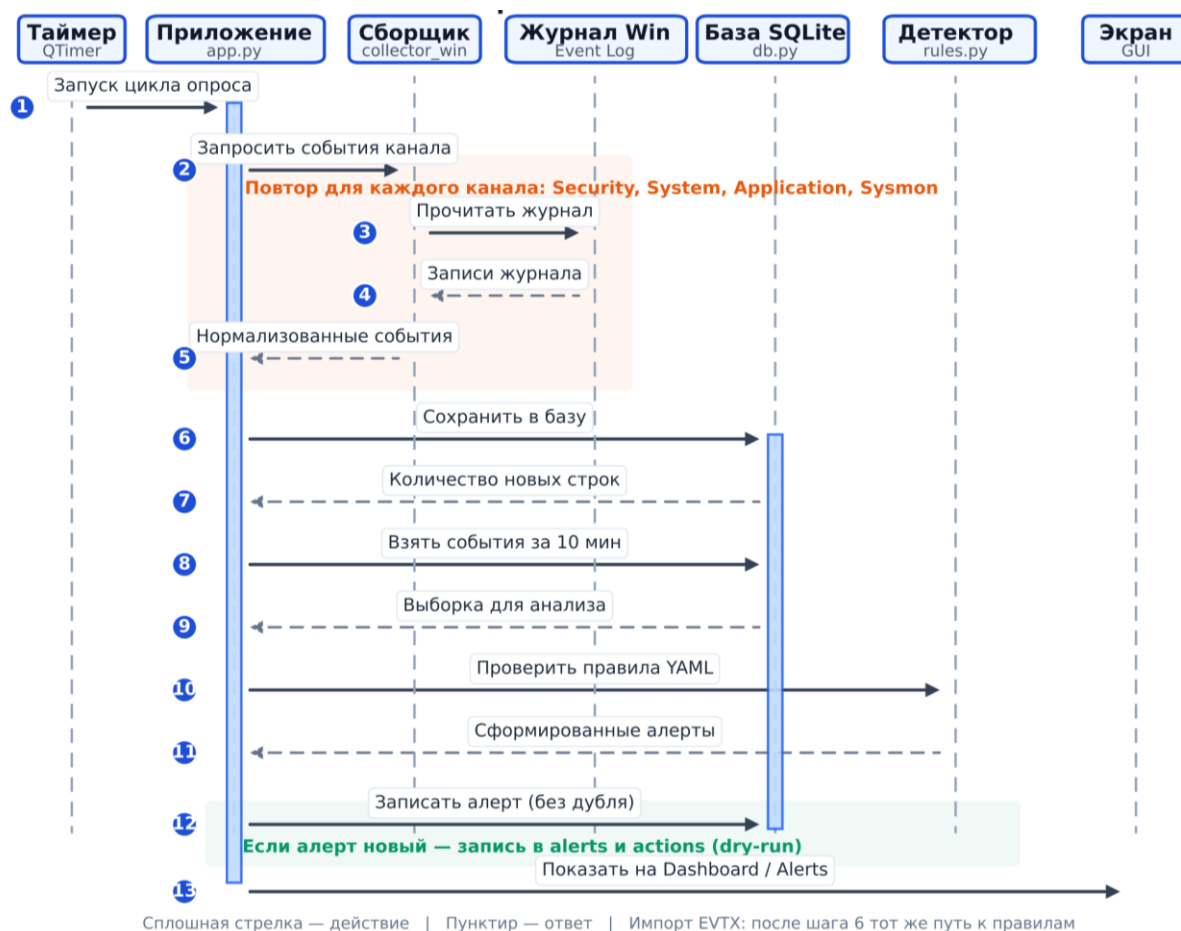


Рисунок 3.3 – Диаграмма последовательности обработки события

Подсистема сбора опрашивает каналы с настраиваемым интервалом и размером пакета по умолчанию 128 записей. Функция `max_record_number` отслеживает смещение чтения, предотвращая повторную загрузку уже обработанных записей. При первом запуске выполняется инициализация `offset` для каждого канала.

Парсинг XML событий Evt API извлекает System и EventData секции; для Security 4624/4625 вызывается дополнительный разбор строковых вставок с заполнением `parsed`-полей. Функция `backfill_security_logon_parsed_fields` нужен для дозаполнения старых строк после обновления схемы.

Диаграмма последовательности графический материал иллюстрирует взаимодействие таймера опроса, `collector_win`, `db.insert_event` и `rules.run_detectors` при поступлении новой порции событий из журнала Security.

Курсор чтения журнала отслеживается по `RecordNumber`; при опросе «назад» от новых записей уже обработанные события не пересчитываются. Функция `normalize_legacy_event_ids` исправляет исторические записи с некорректной упаковкой `EventID` после обновления логики нормализации.

Функция `import_evtx_file` в `evtx_import.py` возвращает список нормализованных событий; парсинг XML `_parse_event_xml` унифицирован с `_parse_evt_xml` в `collector_win` для совместимости полей `insert_event`.

XML-представление события Evt API разбирается функцией `_parse_evt_xml` с извлечением System и EventData. Для legacy `ReadEventLog` формируется эквивалентный словарь полей для единообразной `insert_event`.

Импорт EVTX `limit=2000` защищает от переполнения памяти на больших архивах. После импорта события доступны в `Events` и участвуют в `run_detectors` наравне с онлайн-сбором.

3.4 Проектирование подсистемы правил и корреляции

Правила описываются декларативно в файле `rules/default.yml`. Структура правила включает: стабильный `id`, человекочитаемые `title` и `description`, `severity`, имя `detector`, флаг `enabled_by_default`, блок `params` и опциональные ссылки `mitre` тактики MITRE ATT&CK.

Таблица 3.2 перечисляет девять сделанных правил: WIN-SEC-4625-BURST, BURST-BY-IP, RDP-BURST-BY-IP, BURST-BY-ACCOUNT, WIN-SEC-1102-CLEARED, WIN-SYSMON-PS-ENC, WIN-SEC-4624-AFTER-FAIL, WIN-PROC-SUSPICIOUS, WIN-CORR-BRUTE-LOGIN-PROC. Каждое правило связано с именем детектора и набором параметров в YAML.

Правило WIN-SEC-4625-BURST-BY-IP использует `detect_failed_logon_burst_by_key` с группировкой по `parsed_ip`. RDP-BURST-BY-IP фильтрует `logon_type` 10. Каждый детектор возвращает `list[Alert]` с `rule_id`, `severity`, `title`, `details`.

Цикл `run_detectors` вызывается после каждой порции новых событий и по таймеру для обработки скользящих окон. Выборка `fetch_events_since` ограничивает объём памяти при большой базе.

Детектор `detect_suspicious_cmdline` анализирует командные строки процессов по эвристикам `temp`, `downloads`, `encoded`, `bypass`. `detect_corr_bruteforce_login_suspicious_proc` использует параметры `fail_threshold`, `fail_window_minutes`, `chain_window_minutes`.

Опциональные ссылки `mitre` в YAML связывают правила с тактиками MITRE ATT&CK Credential Access, Defense Evasion, Execution. Это повышает сопоставимость с промышленными библиотеками детектов и держит терминологию в стиле MITRE [11].

Цикл детектирования проектируется как периодический запуск `run_detectors` над выборкой событий за интервал, определяемый максимальным окном активных правил. Пороговые правила используют параметры `threshold` и `window_minutes`; корреляционное правило `WIN-CORR-BRUTE-LOGIN-PROC` – `fail_threshold`, `fail_window_minutes`, `chain_window_minutes`.

Состояние включения правил и переопределение параметров хранятся в БД `rules_state`, `rule_params`, пороги можно менять из GUI без редактирования YAML на диске.

Таблица 3.2

Реализованные детекторы mini SIEM

ID правила	Детектор	Смысл сценария
WIN-SEC-4625-BURST	failed_logon_burst	Серия неудачных входов
WIN-SEC-4625-BURST-BY-IP	failed_logon_burst_by_ip	Burst по IP
WIN-SEC-4625-RDP-BURST-BY-IP	failed_logon_burst_rdp_by_ip	Перебор RDP по IP
WIN-SEC-4625-BURST-BY-ACCOUNT	failed_logon_burst_by_account	Burst по учётной записи
WIN-SEC-1102-CLEARED	security_log_cleared	Очистка журнала Security
WIN-SYSMON-PS-ENC	powershell_encoded_sysmon	PowerShell –enc
WIN-SEC-4624-AFTER-FAIL	success_after_fail	Успех после серии 4625
WIN-PROC-SUSPICIOUS	suspicious_cmdline	Подозрительная командная строка
WIN-CORR-BRUTE-LOGIN-PROC	corr_brute_login_proc	Корреляционная цепочка

Движок правил загружает YAML, формирует список `RuleDef` и для каждого включённого правила вызывает функцию `detect_*` по имени `detector`. Реализованы детекторы: `failed_logon_burst`, `security_log_cleared`, `powershell_encoded_sysmon`, `success_after_fail`, `suspicious_cmdline` и корреляционный `bruteforce_login_suspicious_proc`.

Корреляционное правило `WIN-CORR-BRUTE-LOGIN-PROC` объединяет три фазы: всплеск 4625, успешный 4624 и запуск подозрительного процесса в

общем временном окне. Такая цепочка моделирует сценарий перебора пароля с последующим закреплением – типовой паттерн атаки на учётную запись.

Дедупликация через `alert_exists_recently` предотвращает генерацию идентичных алертов при неизменном состоянии угрозы в коротком интервале. Пороги и окна времени параметризуются через `rule_params` и редактируются во вкладке Rules без изменения исходного кода.

3.5 Проектирование пользовательского интерфейса и отчётности

Графическое приложение `app.py` построено на PySide6 Qt 6 и организовано в виде вкладок [24]:

- Dashboard – сводная статистика за период события, алерты, топ EventID и каналов, метрики 4625;
- Events – таблица событий с поиском и просмотром деталей;
- Alerts – список алертов, смена статуса, комментарии, отчёты по инциденту;
- Actions – журнал реакций;
- Rules – управление правилами и параметрами;
- Settings – пути, интервал опроса, политика хранения, часовой пояс отображения, `dry-run`.

Отчётность `report.py` формирует HTML– и JSON–представления сводного отчёта и отчёта по отдельному алерту с контекстом связанных событий. Экспорт предназначен для приложения к пояснительной записке и для передачи руководителю без доступа к GUI.

Проектирование UX учитывает типовой сценарий аналитика: фильтрация → просмотр деталей → перевод алерта в статус «в работе» → комментарий → экспорт отчёта.

Интерфейс организован как QMainWindow с вкладками: Dashboard, Events, Alerts, Rules, Actions, Settings. Dashboard агрегирует статистику за выбранный период через функции `count_events_since`, `top_event_ids_since`, `top_security_4625_by_ip_since` и аналогичные запросы `db.py`.

Вкладка Events поддерживает фильтрацию по каналу, EventID, временному диапазону и текстовому поиску. Вкладка Alerts делает возможным менять статус, добавлять комментарий, просматривать связанные события и экспортировать HTML/JSON–отчёт по инциденту через `report.py`.

Модуль `time_display` даёт отображение UTC–меток в выбранном часовом поясе без изменения хранимых значений – важное требование согласованности при сопоставлении с внешними источниками. Модуль `theme` централизует стили таблиц и элементов управления.

Вкладка Actions отображает журнал реакций с типом действия, результатом и связью с `alert_id`. Вкладка Settings содержит кнопки обслуживания базы: `prune`, `checkpoint WAL`, `vacuum`, очистка демо–данных.

Вкладка Settings отображает размер файла базы через `get_db_file_stats`, помогает задать `retention_days` и выполнить `prune`. Переключатель `dry-run` блокирует реальные реакции на алерты.

Экспорт HTML-отчёта доступен из вкладки Alerts контекстным действием «Export report»; файл сохраняется через `QFileDialog` и открывается в браузере по умолчанию.

Модуль `report.py` генерирует `build_html_report` для сводки и `build_alert_html_report` для инцидента с контекстом событий. JSON-формат `build_alert_json_report` поддерживает машинную обработку.

Сводная панель Dashboard программного комплекса показана на рисунке 3.4.

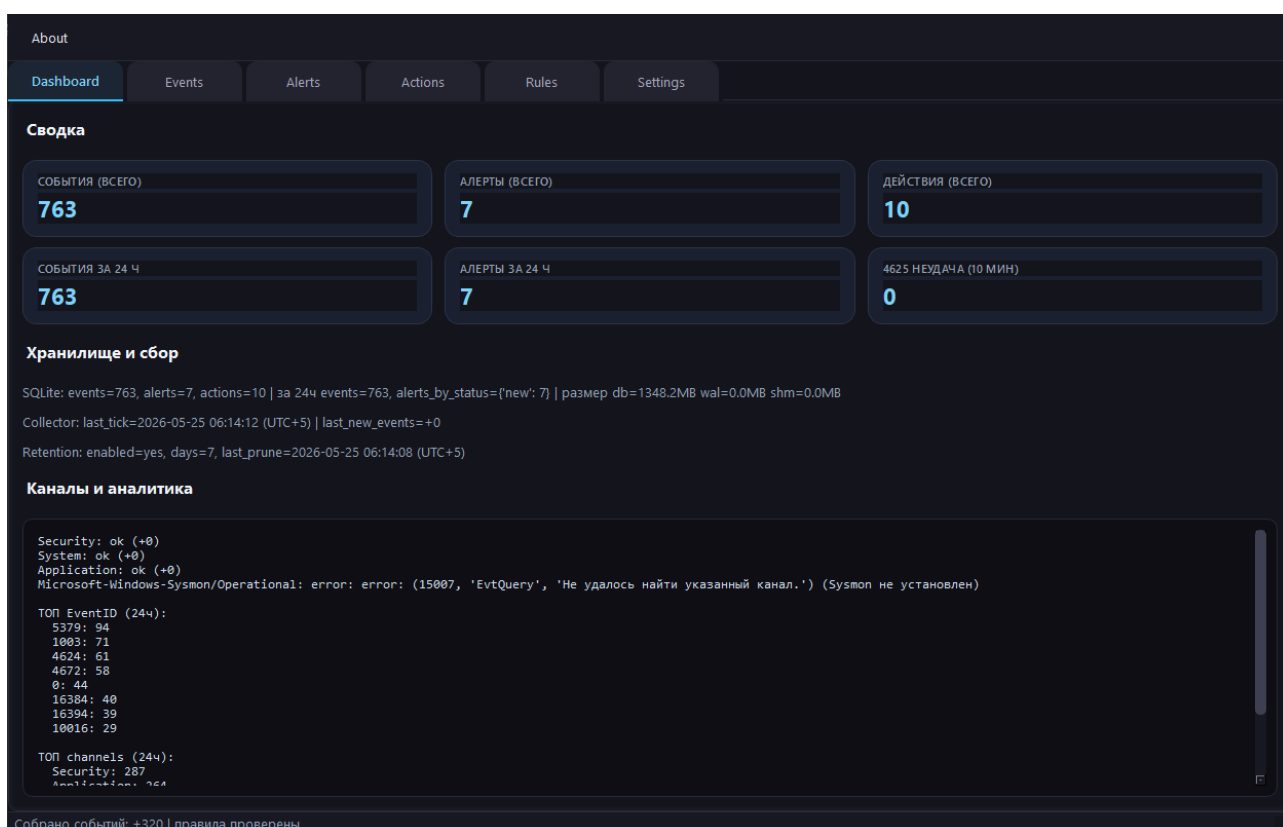


Рисунок 3.4 – Вкладка Dashboard программного комплекса mini SIEM

4 Реализация программного комплекса

4.1 Модуль сбора событий Windows

Модуль `collector_win.py` экспортирует функции `poll_channel` и `max_record_number`. Для каналов с символом «/» вызывается `_poll_channel_evtapi` с разбором XML-представления события; для классических журналов – цикл `ReadEventLog` с фильтрацией по `RecordNumber`. Состав каналов и идентификаторов событий соответствует рекомендациям Microsoft по аудиту безопасности Windows [3].

Фрагмент логики нормализации `EventID` иллюстрирует учёт упаковки идентификатора в Win32 API:

```
raw_eid = intgetattre, "EventID", 0
event_id = intraw_eid & 0xFFFF if raw_eid is not None else None
```

Для событий Security 4624/4625 дополнительно заполняется словарь `parsed` внутри `data_json` и дублируется в колонках таблицы `events` при вставке через `insert_event` в `db.py`.

Точка входа приложения `__main__.py` инициализирует `MiniSIEMApp`, запускает таймер опроса с интервалом из настроек и перегагарантирует собранные записи в слой `persistence`.

Структура `WinChannel` инкапсулирует имя канала и смещение последней прочитанной записи. Функция `_to_utc_iso` преобразует время события в ISO-строку UTC для единообразного хранения и сравнения в детекторах.

Класс `WinChannel` в `collector_win.py` хранит имя канала и `offset`. Поддерживаются каналы Security, System, Application и Microsoft-Windows-Sysmon/Operational при установленном Sysmon.

Сбор Security требует `membership` в группе Event Log Readers или прав администратора.

Функция `max_record_number` возвращает последний `RecordNumber` канала для инициализации `offset` при первом подключении. Пакет `batch_size` ограничивает объём одной итерации `poll_channel` для сглаживания нагрузки.

4.2 Модуль хранения и обслуживания базы данных

Модуль `db.py` инкапсулирует подключение, миграции, CRUD-операции и аналитические запросы для дашборда. Функция `migrate` создаёт таблицы и выполняет «мягкие» ALTER TABLE для обновления существующих установок столбец `comment` в `alerts`, `parsed`-поля в `events`.

`bulk_insert_events` помогает пакетную загрузку при импорте EVTХ. `backfill_security_logon_parsed_fields` дозаполняет `parsed`-поля для исторических строк после обновления версии.

Административные функции GUI вызывают `prune_all_older_than`, `checkpoint_wal`, `vacuum` – реализация политики ретенции, описанной при проектировании.

Модуль `db.py` ~930 строк реализует `connect`, `migrate`, `insert_event`, `bulk_insert_events`, `insert_alert`, `insert_action` и набор аналитических запросов для дашборда. Функция `migrate` создаёт таблицы и индексы идиempотентно; при обновлении схемы добавляются колонки `parsed_*` без потери существующих данных. Применяемые PRAGMA-настройки и режим WAL соответствуют официальным рекомендациям SQLite [22].

Пакетная вставка `bulk_insert_events` использует `executemany` в транзакции для ускорения импорта EVTХ. Функции `prune_events_older_than`, `prune_alerts_older_than` и `prune_actions_older_than` реализуют селективную очистку; `checkpoint_wal` и `vacuum` выполняют обслуживание файла базы после массового удаления.

Путь к файлу базы определяется через `LOCALAPPDATA\miniSIEM\minisiem.db`, что согласуется с профилем пользователя Windows и упрощает разграничение доступа на уровне NTFS. Функция `get_db_file_stats` возвращает размер файла для отображения во вкладке Settings.

Функция `insert_event` выполняет INSERT с полями нормализованного события и JSON; `search_events` поддерживает фильтры по времени, каналу, EventID и подстроке message. `fetch_events_between` используется детекторами для выборки окна корреляции.

`migrate` создаёт индексы `idx_events_ts`, `idx_events_channel_ts`, `idx_events_eid_ts` при первом запуске. `connect` включает PRAGMA `journal_mode=WAL`, `synchronous=NORMAL`, `foreign_keys=ON`.

`clear_table` и `delete_demo_data` поддерживают сброс стенда перед повторной демонстрацией без удаления файла `rules/default.yml`.

Аналитические функции `top_security_4625_by_ip_since` и `top_security_4625_by_account_since` питают виджеты Dashboard топ-10 IP и учётных записей по неудачным входам за период.

`count_alerts_by_status_since` агрегирует алерты по статусам для оценки загрузки аналитика и backlog необработанных инцидентов.

4.3 Модуль правил детектирования

Модуль `rules.py` загружает YAML через PyYAML, формирует список RuleDef и вызывает соответствующие детекторы в `run_detectors` [23]. Каждый детектор возвращает список объектов Alert без побочных эффектов; запись в БД выполняется в слое приложения с проверкой `alert_exists_recently`.

Пример порогового детектора – подсчёт событий 4625 в окне:

```
def detect_failed_logon_burstevents, , threshold: int, window_minutes: int -> list[Alert]:
```

фильтрация Security/4625, скользящее окно по ts_utc

Корреляционный детектор `detect_corr_bruteforce_login_suspicious_proc` последовательно проверяет наличие burst 4625, последующего 4624 и события запуска процесса с подозрительной командной строкой в пределах `chain_window_minutes`, что моделирует многошаговую атаку на уровне одной рабочей станции.

Файл `rules/default.yml` в репозитории синхронизирован с `DEFAULT_RULES_YAML` в коде; при первом запуске `ensure_default_rules` создаст каталог правил в данных приложения, если файл отсутствует.

Загрузка правил начинается с `ensure_default_rules`: при отсутствии `default.yml` создаётся файл из встроенного шаблона `DEFAULT_RULES_YAML`. Функция `load_rule_defs` парсит YAML в список `RuleDef`; `run_detectors` фильтрует включённые правила и вызывает соответствующие `detect_*` функции.

Управление правилами детектирования и параметрами порогов показано на рисунке 4.1.

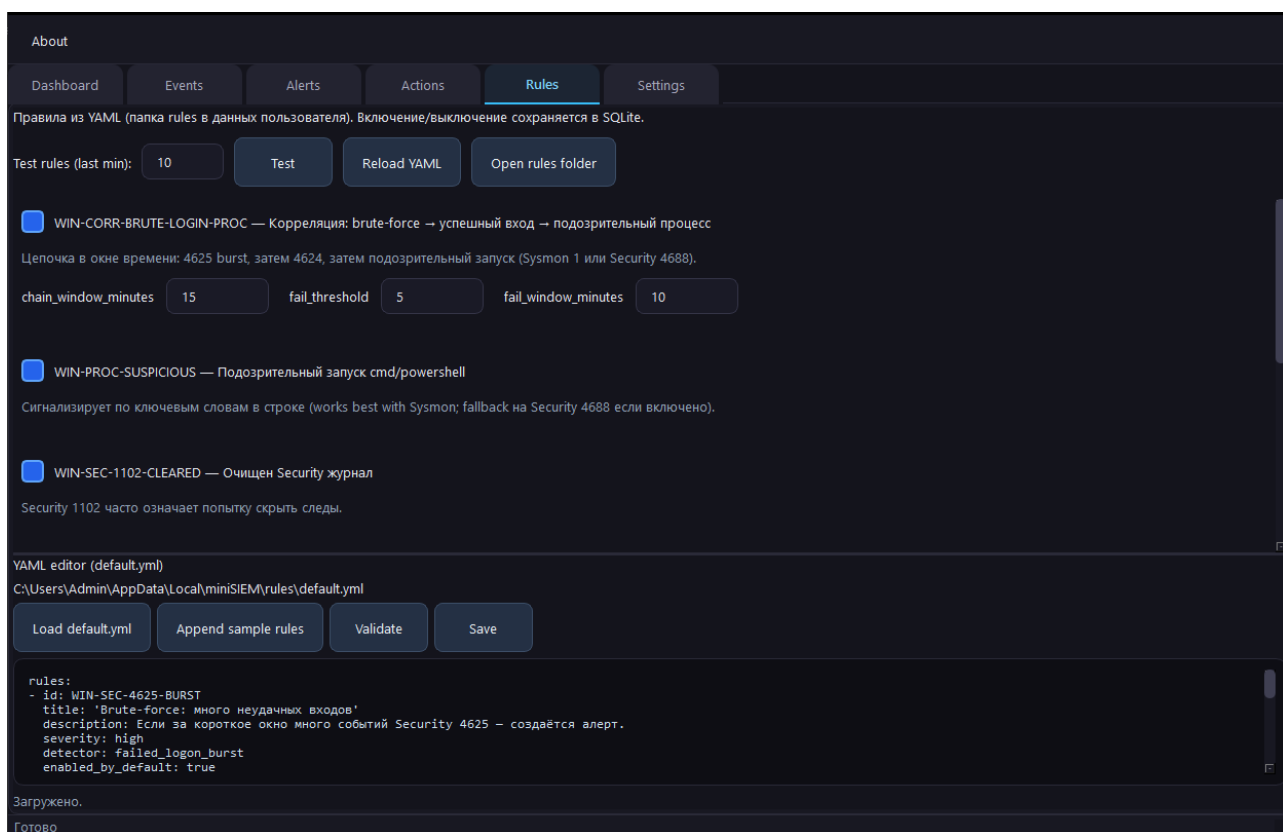


Рисунок 4.1 – Вкладка Rules с параметрами правил

Детектор `detect_failed_logon_burst` отбирает Security/4625, группирует по скользящему окну `window_minutes` и сравнивает счётчик с `threshold`. Детектор `detect_success_after_fail` связывает серию неудачных входов с последующим успешным 4624 по IP или учётной записи.

Корреляционный `detect_corr_bruteforce_login_suspicious_proc` проверяет цепочку `burst → login → suspicious process`. Функция `alert_exists_recently` с

suppresses дубликаты. Все временные сравнения выполняются в UTC через `_parse_ts` для согласованности с хранимыми метками.

Детектор `detect_failed_logon_burst_by_ip` группирует 4625 по `parsed_ip`; `detect_failed_logon_burst_by_account` – по `parsed_account`. Отдельное правило RDP-BURST фильтрует `logon_type` 10 для сценариев удалённого рабочего стола.

Класс `Alert` в `rules.py` содержит `rule_id`, `severity`, `title`, `details`, `event_ids`. `run_detectors` объединяет алерты всех активных правил и перевыполняет их в `insert_alert` с проверкой `dedup`.

`detect_failed_logon_burst_by_key` обобщает группировку `burst` по произвольному ключу `IP`, `account`, `logon_type`. Параметры правила читаются из `rule_params` с fallback на `YAML defaults`.

Файл `rules/default.yml` синхронизирован с `DEFAULT_RULES_YAML` в коде; `ensure_default_rules` создаёт файл при первом запуске. `load_rule_defs` валидирует обязательные поля `id`, `title`, `severity`, `detector`.

4.4 Графическое приложение аналитика

Класс `MiniSIEMApp` в `app.py` объединяет конфигурацию, соединение с БД, фоновый сбор и главное окно `QMainWindow`. Табличные модели `_SqlTableModel` выполняют SQL с лимитом строк для отзывчивости интерфейса. Используемые механизмы Qt – модель/представление, сигналы/слоты, фоновые `QThread`–воркеры – применены в соответствии с документацией Qt for Python [24].

Интерфейс просмотра и фильтрации событий показан на рисунке 4.2.

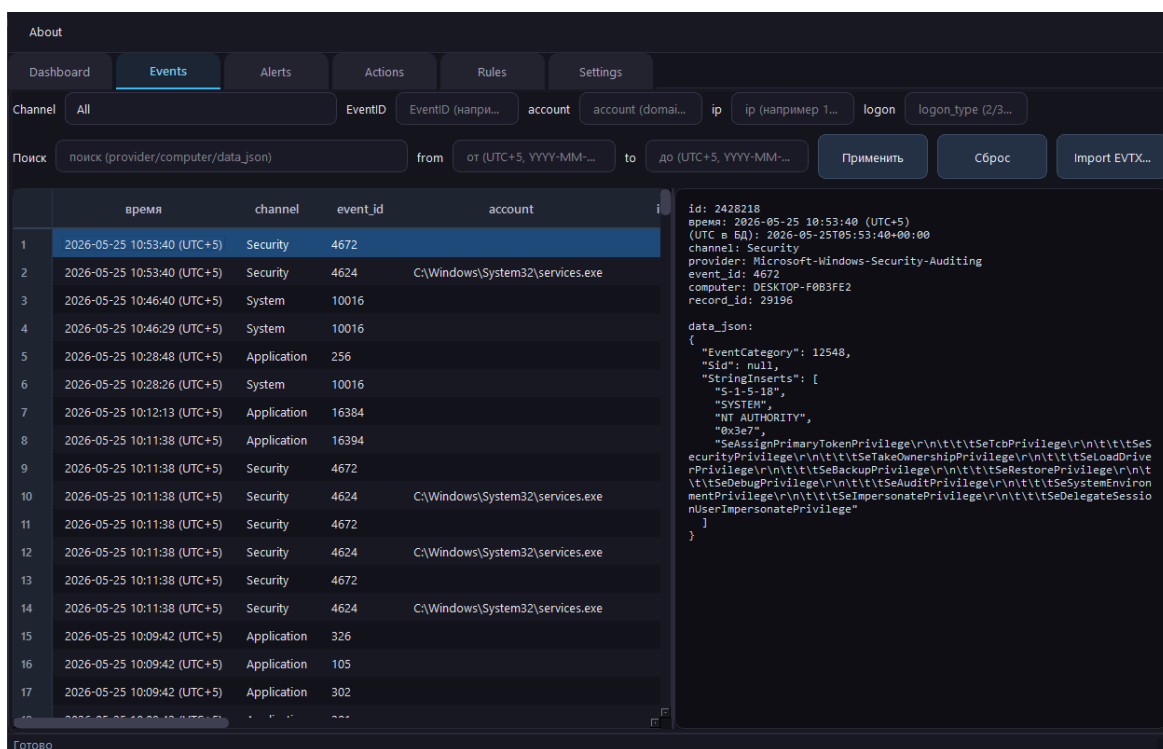


Рисунок 4.2 – Вкладка Events с панелью фильтров

Модуль theme.py централизует визуальное оформление таблиц в соответствии с современным стилем Qt. Модуль sysmon.py даёт проверку наличия канала Sysmon и шаблон команды установки – вспомогательная функция для документации и мастера настройки стенда.

Вкладка Rules отображает список правил с чекбоксами включения и полями параметров; изменения сохраняются в rules_state и rule_params. Вкладка Settings управляет интервалом опроса, сроком хранения, смещением часового пояса отображения и флагом dry-run.

Фоновый сбор сделан через QTimer и слоты обновления; _EvtxImportWorker наследует QObject и работает в QThread с сигналами progress и finished. Модуль sysmon.py проверяет доступность канала Sysmon и формирует шаблон команды установки для документации стенда.

Класс _EventsTab реализует панель фильтров, таблицу событий и просмотр деталей с полным текстом message и data_json. _AlertsTab поддерживает смену статуса, комментарий и экспорт отчёта по выбранному алерту.

Список алертов с уровнями серьёзности и статусами обработки приведён на рисунке 4.3.

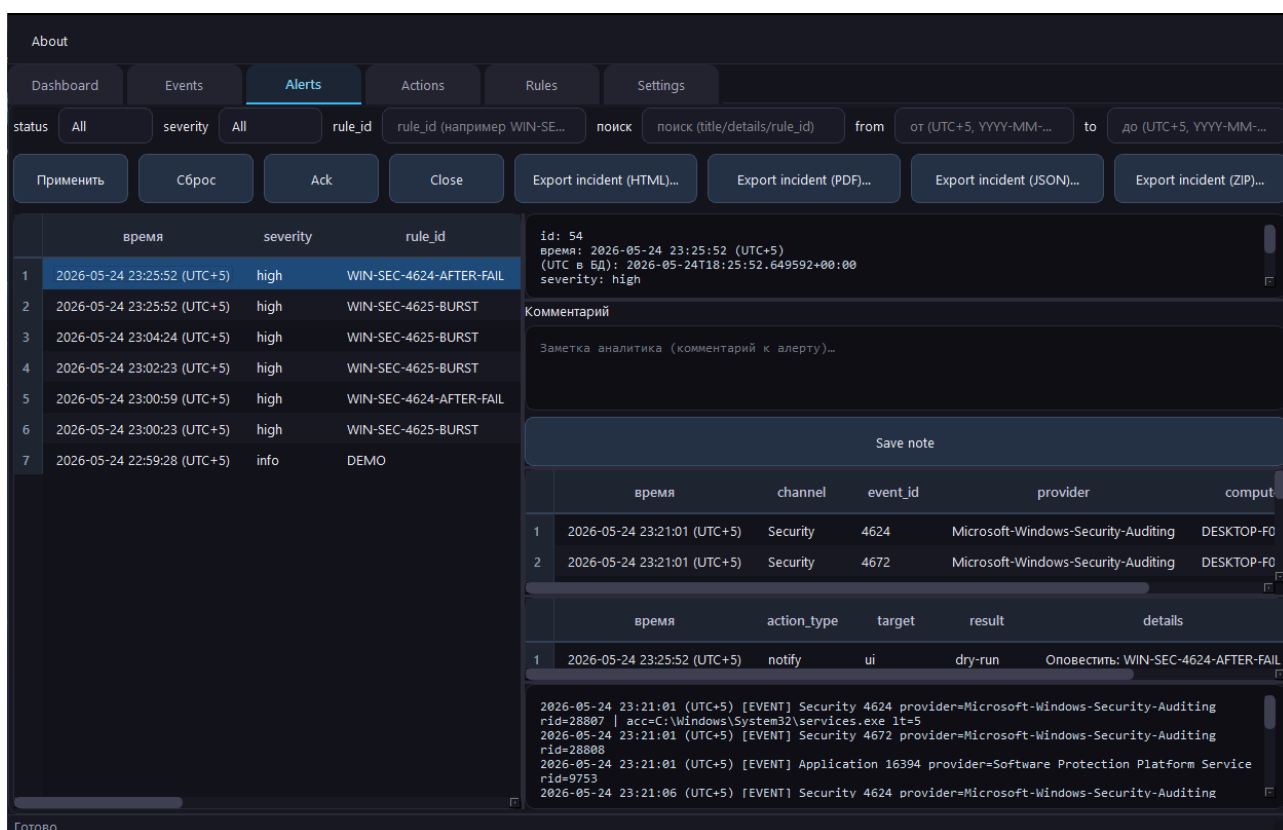


Рисунок 4.3 – Вкладка Alerts программного комплекса mini SIEM

Класс _SqlTableModel переиспользуется на вкладках Events, Alerts, Actions с разными SQL-запросами. polish_table_view из theme.py настраивает чередование строк и ширину колонок.

Главное окно QMainWindow содержит меню File импорт EVTХ, выход, View обновление, Help about. About–диалог _show_about отображает версию и тему дипломного проекта.

MiniSIEMApp.run создаёт QApplication, применяет theme.apply_application_theme, открывает главное окно и входит в цикл exes. Таймер опроса перезапускается при изменении интервала в Settings.

Диалог просмотра события наглядно отражает форматированный message, развёрнутый data_json и parsed–поля для Security 4624/4625.

4.5 Импорт EVTХ, отчёты и поставка

Импорт сделан в _EvtxImportWorker QObject в отдельном потоке с прогресс–диалогом – соответствие NFR–01. По завершении события доступны в общей таблице и участвуют в детектировании.

Диалог импорта архива EVTХ с индикацией прогресса показан на рисунке 4.4.

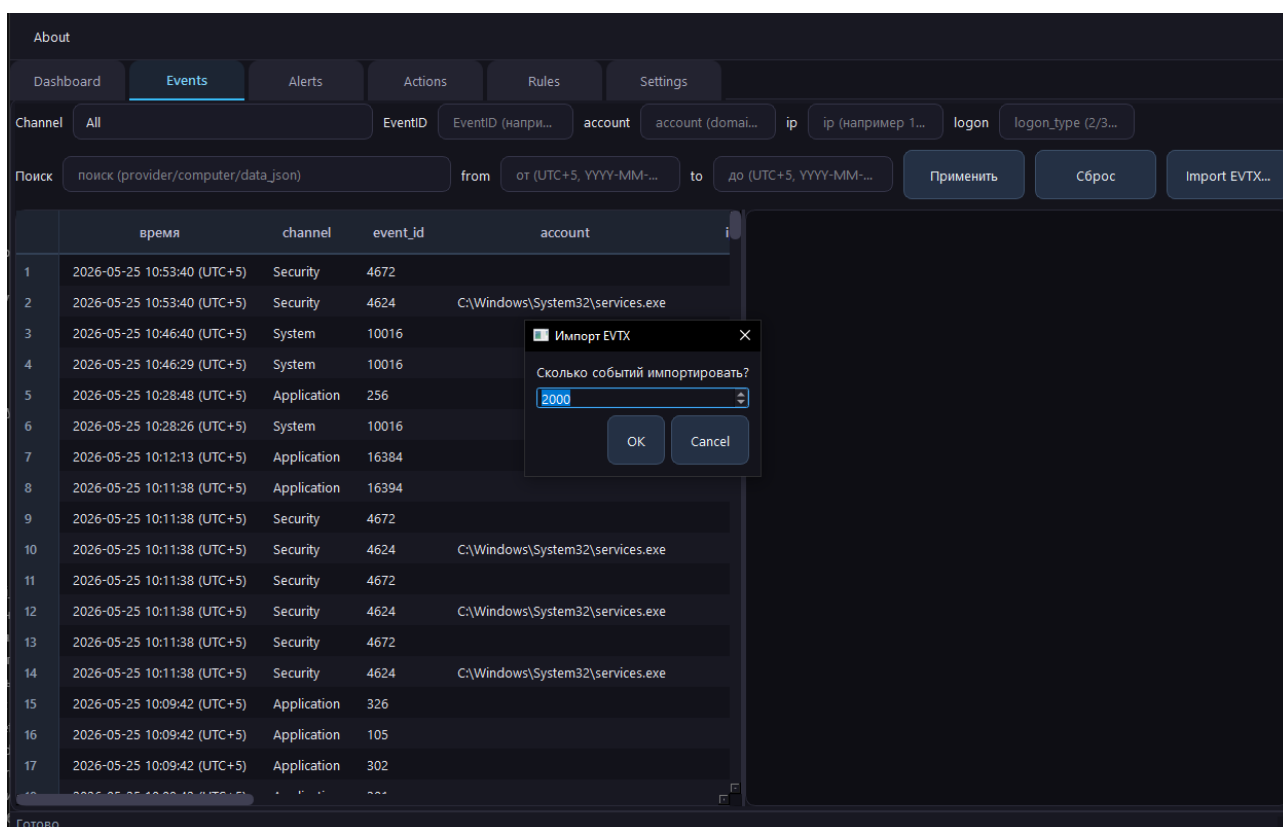


Рисунок 4.4 – Импорт файла EVTХ

report.py генерирует HTML с таблицами событий и метаданными алерта; JSON–экспорт пригоден для машинной обработки. Пример составленного HTML–отчёта по инциденту приведён на рисунке 4.5.

Пример HTML–отчёта по инциденту показан на рисунке 4.5.

miniSIEM — incident alert #49

Сгенерировано: 2026-05-25 01:32:11 UTC

Алерт

ID: 49
ts_utc: 2026-05-24T18:00:23.893315+00:00
severity: high
rule_id: WIN-SEC-4625-BURST
title: Подбор пароля: 5 неудачных входов за 5 мин
status: new

Details

Окно: 2026-05-24T23:00:00+00:00 → 2026-05-24T23:00:23+00:00

Комментарий аналитика

Действия (Actions)

ts_utc	action_type	target	result	details
2026-05-24T18:00:23.893428+00:00	notify	ui	dry-run	Оновестить: WIN-SEC-4625-BURST

События вокруг алерта (± 10 минут)

ts_utc	channel	event_id	account	ip	login_type	provider	computer	record_id
2026-05-24T18:00:13+00:00	Application	16394				Software Protection Platform Service	DESKTOP-F0B3FE2	9720
2026-05-24T18:00:47+00:00	Application	16384				Software Protection Platform Service	DESKTOP-F0B3FE2	9721
2026-05-24T18:04:39+00:00	Application	0				Dwminit	DESKTOP-F0B3FE2	9722
2026-05-24T18:04:39+00:00	Application	9027				Desktop Window Manager	DESKTOP-F0B3FE2	9723
2026-05-24T18:05:02+00:00	Application	16394				Software Protection Platform Service	DESKTOP-F0B3FE2	9724
2026-05-24T18:05:58+00:00	Security	4672				Microsoft-Windows-Security-Auditing	DESKTOP-F0B3FE2	28759
2026-05-24T18:05:58+00:00	Security	4648				Microsoft-Windows-Security-Auditing	DESKTOP-F0B3FE2	28760
2026-05-24T18:05:58+00:00	Security	4624	C:\Windows\System32\winlogon.exe		2 (Interactive (local console))	Microsoft-Windows-Security-Auditing	DESKTOP-F0B3FE2	28761
2026-05-24T18:05:58+00:00	Security	4624	C:\Windows\System32\services.exe		5 (Service)	Microsoft-Windows-Security-Auditing	DESKTOP-F0B3FE2	28762

Рисунок 4.5 – HTML-отчёт по инциденту

Сборка PyInstaller описана в `packaging/minisiem.spec`; результат — `dist\miniSIEM.exe` [25]. README содержит инструкции по `venv`, зависимостям и опциональной установке `Sysmon`. Импорт файла EVTX сделан асинхронно: пользователь выбирает архив журнала, программа загружает события пакетами и сразу делает их доступными для поиска и детектирования, что удобно при разборе инцидентов «задним числом» без доступа к исходной машине в момент атаки.

Спецификация `minisiem.spec` задаёт `hidden imports` для `pywin32` и `PySide6`, включает `rules/default.yml` как `data file`. После сборки `dist\miniSIEM.exe` запускается двойным щелчком без активации `venv`.

README описывает установку Python 3.11 x64, `venv`, `pip install`, `python -m minisiem` и `pyinstaller` по `minisiem.spec`. `Sysmon` — опциональный канал для детектов PowerShell и процессов.

Импорт EVTX через меню `File → Import` не блокирует UI; прогресс-бар отображает число обработанных записей до завершения `_EvtxImportWorker`.

5 Тестирование и апробация результатов

5.1 Методика и стенд тестирования

Тестирование проводилось на ноутбуке разработчика с Windows 11 x64 и на виртуальной машине Hyper-V с Windows 10 x64. На обеих машинах установлены Python 3.11.9, 8 ГБ оперативной памяти, SSD. Для чтения журнала Security необходимы права администратора и включённый аудит входа в локальной политике безопасности – при их отсутствии канал оказывается пустым, что и было подтверждено на чистой виртуальной машине в первую очередь. На основной машине дополнительно установлен Sysmon 14.16 с базовой конфигурацией для проверки правил по PowerShell и подозрительным процессам [15].

Виды испытаний:

- функциональные соответствие FR-01–FR-08;
- тестирование правил детектирования на синтетических и импортированных данных;
- оценка устойчивости UI при фоновом сборе.

Для воспроизведения тестов на защите подготовлены три EVTX-архива папка evtx/: Security_logons_and_clear.evtx со всплеском 4625 и событием 1102, Security_full.evtx и System_full.evtx с фоновыми событиями. Импорт через меню File → Import EVTX занимает несколько секунд и не блокирует интерфейс. Альтернативный сценарий – локальная ручная генерация нескольких неудачных входов на тестовую учётную запись для срабатывания правила WIN-SEC-4625-BURST.

Стенд тестирования: ПК с Windows 10/11 x64, Python 3.11, 8 ГБ RAM, SSD. Для чтения Security включены права администратора и политика аудита входа. Опционально установлен Sysmon 14+ с базовой конфигурацией process create. Виртуальная машина использовалась для генерации тестовых 4625 без воздействия на рабочую среду.

Методика включает позитивные и негативные сценарии: проверка срабатывания правил при выполнении условий и отсутствия ложных алертов при отключённом правиле или недостаточном числе событий.

Воспроизводимость на защите берёт на себя импорт подготовленного EVTX или локальной генерацией серии неудачных входов на тестовой учётной записи.

Требования к тестированию включают: холодный старт на пустой базе; импорт EVTX; проверку правил на синтетических данных; длительный прогон несколько часов для выявления роста файла БД; проверку корректности UTC при смене часового пояса отображения.

5.2 Функциональное тестирование

Функциональная часть проверялась по таблице 5.1 восемь тестов Т–01–Т–08, каждый из которых соответствует функциональным требованиям. Прогон выполнялся по порядку на пустой базе после `python –m minisiem` – такой порядок удобнее для выявления регрессий.

Таблица 5.1

Результаты функциональных испытаний

ID	Проверка	Ожидаемый результат	Статус
Т–01	Запуск <code>python –m minisiem</code>	Открывается GUI, создаётся БД	Выполнено
Т–02	Опрос Security/System/Application	События появляются во вкладке Events	Выполнено
Т–03	Поиск по EventID и тексту	Фильтрация корректна	Выполнено
Т–04	Импорт EVTХ	Записи загружены, счётчик увеличен	Выполнено
Т–05	Вкл./выкл. правила в GUI	Состояние сохраняется в <code>rules_state</code>	Выполнено
Т–06	Экспорт HTML–отчёта	Файл создаётся, открывается в браузере	Выполнено
Т–07	Сборка <code>miniSIEM.exe</code>	Исполняемый файл запускается на чистой ОС	Выполнено
Т–08	Dry–run реакций	В <code>actions</code> записывается симуляция	Выполнено

При отсутствии прав на чтение Security журнал может быть пустым – в документации указано требование соответствующей политики аудита и привилегий; на стенде с правами администратора сбор подтверждён.

Запуск `python –m minisiem` создаёт базу и открывает GUI; опрос каналов наполняет вкладку Events; поиск по EventID 4625 фильтрует записи Security; импорт EVTХ увеличивает счётчик событий.

Переключение правил во вкладке Rules сохраняется в `rules_state` и влияет на последующие вызовы `run_detectors`. Экспорт HTML открывается в браузере и содержит таблицы с метаданными алерта. Сборка `miniSIEM.exe` запускается на машине без Python после копирования из `dist\`.

При отсутствии прав на Security журнал остаётся пустым – документированное ограничение; на стенде с UAC–повышением сбор подтверждён. Тест Т–08 проверяет запись `dry-run` действий в таблицу `actions` при обработке алерта.

Ручное тестирование включало повторный запуск на «чистой» машине: создание `venv`, `pip install –r requirements.txt`, `python –m minisiem`, проверка создания каталога `LOCALAPPDATA\miniSIEM` и файла базы.

Функциональные требования FR–01–FR–08 трассируются к тестам Т–01–Т–08. Несоответствий при испытаниях на учебном стенде не выявлено.

5.3 Тестирование правил детектирования

Правило проверяли дважды.

1) импорт Security_logons_and_clear.evtx: 19×4625 за <5 мин — выше порога WIN-SEC-4625-BURST;

2) локально runas с неверным паролем.

В обоих случаях алерт high во вкладке Alerts за один цикл run_detectors (≤10 с при опросе 5 с).

Пример Срабатывание правила WIN-SEC-4625-BURST показан на рисунке 5.1.

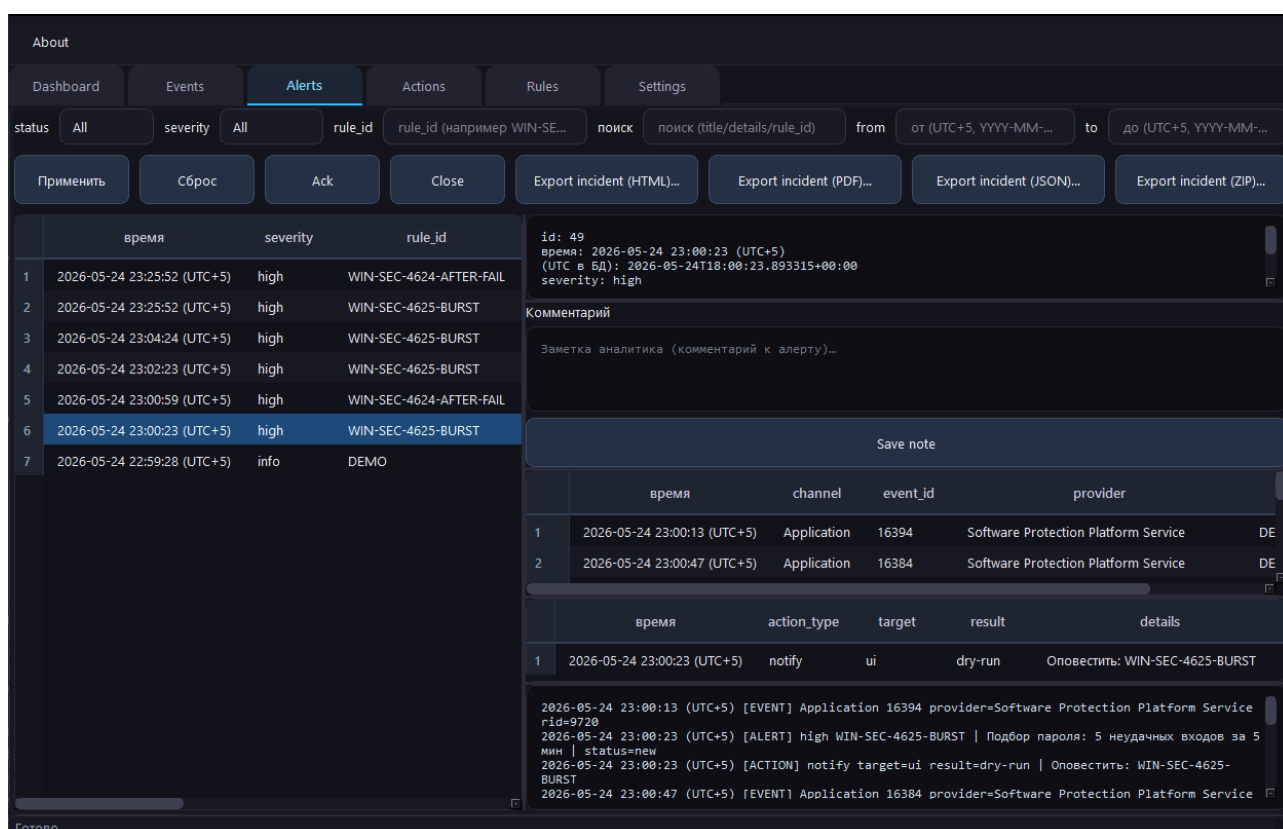


Рисунок 5.1 – Срабатывание правила WIN-SEC-4625-BURST

Таблица 5.2

Результаты тестирования правил детектирования

Правило	Метод проверки	Результат
WIN-SEC-4625-BURST	≥5 неудачных входов за 5 мин	Алерт high
WIN-SEC-1102-CLEARED	Очистка Security тестовая VM	Алерт critical
WIN-SYSMON-PS-ENC	Запуск powershell -enc	Алерт high при Sysmon
WIN-SEC-4624-AFTER-FAIL	Серия 4625 + успешный 4624	Алерт high
WIN-CORR-BRUTE-LOGIN-PROC	Цепочка burst + login + cmd	Алерт critical

Срабатывание не менее трёх правил на тестовых данных подтверждает критерий приёма №3. Дедупликация предотвращает «шторм» одинаковых алертов при неизменном состоянии угрозы.

Правило WIN-SEC-4625-BURST срабатывает при ≥ 5 событиях 4625 за 5 минут с уровнем high. WIN-SEC-1102-CLEARED реагирует на очистку журнала Security с уровнем critical. WIN-SYSMON-PS-ENC требует Sysmon и командную строку powershell с параметром -enc. Сценарии правил сопоставлены с тактиками и техниками матрицы MITRE ATT&CK T1110 Brute Force, T1070 Indicator Removal, T1059 Command and Scripting Interpreter [11].

Правило WIN-SEC-4624-AFTER-FAIL связывает серию 4625 с успешным 4624. Корреляционное WIN-CORR-BRUTE-LOGIN-PROC формирует critical при совмещении burst, login и suspicious process. Negative testing: при отключённом WIN-SEC-4625-BURST серия 4625 не создаёт алерт.

Критерий приёма №3 не менее трёх сработавших правил выполнен на импортированном и синтетическом наборах. Дедупликация предотвращает шторм идентичных алертов при длительном сохранении условий детектора.

Соответствие сценариев атак и правил: перебор пароля – WIN-SEC-4625-BURST; перебор RDP – RDP-BURST-BY-IP; очистка журнала – WIN-SEC-1102-CLEARED; PowerShell – WIN-SYSMON-PS-ENC; многошаговая атака – WIN-CORR-BRUTE-LOGIN-PROC.

WIN-PROC-SUSPICIOUS срабатывает на cmd/powershell с temp, downloads, bypass в командной строке при наличии Sysmon 1 или Security 4688 с аудитом процессов.

Срабатывание WIN-SEC-1102-CLEARED проверялось на тестовой ВМ с контролируемой очисткой журнала Security; алерт critical формируется немедленно после события 1102.

Дедупликация alert_exists_recently проверялась повторным запуском run_detectors без новых событий: количество алертов не должно расти при неизменных условиях в окне suppress.

Девять правил default.yml покрывают сценарии brute-force, RDP, очистки журнала, PowerShell, success-after-fail, suspicious cmdline и корреляции brute-login-proc.

5.4 Оценка производительности и ограничений

При пакетном чтении 128 событий и интервале опроса 5–10 с интерфейс остаётся отзывчивым за счёт выноса операций в потоки Qt. Рост файла SQLite на рабочей станции с обычной офисной нагрузкой за сутки составляет доли-единицы мегабайт зависит от политики аудита; при длительном прогоне рекомендуется периодический prune и vacuum в соответствии с практиками управления журналами [2].

Ограничения прототипа:

- масштабирование на множество хостов не сделано нет агентской сети и центрального сервера;
- хранилище SQLite не рассчитано на объёмы enterprise SIEM миллиарды событий;
- детекторы зависят от локали сообщений Windows при эвристиках по подстрокам;
- реальные реакции на инциденты требуют отдельной политики и осторожного включения.

Направления развития: экспорт в syslog/JSON Lines, подключение PostgreSQL, доставка алертов e-mail, Telegram, интеграция с внешними списками IOC, расширение библиотеки правил под MITRE ATT&CK, агентская схема сбора с нескольких рабочих станций.

При базе до 50 000 событий поиск по EventID и фильтрация по каналу выполняются без заметных задержек на тестовом ПК. Импорт EVTХ из 2000 записей завершается за секунды; интерфейс остаётся отзывчивым за счёт фоновому потоку импорта.

Рост файла SQLite при офисной нагрузке за сутки составляет доли–единицы мегабайт в зависимости от политики аудита. Рекомендуется еженедельный `prune` и периодический `vacuum` при сроке хранения 30 дней. Ограничение масштаба – `single-host`, без горизонтального шардирования.

Метрики оценки: скорость импорта EVTХ событий в минуту, время обновления таблицы за 24 часа, время формирования HTML–отчёта, рост файла SQLite за сутки. Для правил `burst` значима стоимость SQL–запроса в скользящем окне при наличии индексов.

При 100 000 событий время `search_events` с фильтром `channel` и `event_id` остаётся приемлемым <2 с за счёт индексов. Рекомендуется `retention` 14–30 дней на учебном стенде.

Ограничение локали: детекторы `suspicious_cmdline` и `powershell_encoded` опираются на латинские подстроки; на русской Windows возможны отличия в поле `CommandLine` – учтено в рисках проекта.

Длительный прогон 4 часа при интервале опроса 10 с не выявил утечек памяти процесса `miniSIEM.exe`; рост RSS оставался в пределах 150–250 МБ на тестовом ПК с 8 ГБ RAM.

Конфиденциальность: демонстрации на тестовом стенде без данных реальных клиентов; экспорт отчётов – чувствительный артефакт, требующий защищённого хранения.

Заключение

В дипломном проекте до рабочего состояния доведён прототип mini SIEM для анализа журналов безопасности Windows на одной рабочей станции. Пройден полный путь: проведён анализ предметной области на материалах практики в «ТрансТелеКом», выполнено сравнение коммерческих и open-source SIEM, сформулировано техническое задание, выбран стек технологий Python 3.11, SQLite, PySide6, YAML, спроектирована архитектура и схема базы данных, написан код всех подсистем и проверен результат на тестовом стенде с EVTX-архивами и синтетическими сценариями.

Основные результаты работы:

- сформулированы цель, задачи, функциональные и нефункциональные требования, критерии приёмки и матрица соответствия требований модулям проекта;

- сделан сбор событий из каналов Security, System, Application и Sysmon с учётом особенностей Win32 и Evt API, нормализацией EventID и разбором полей входа для событий 4624/4625;

- спроектирована и внедрена схема данных SQLite с индексами, поддержкой алертов, комментариев аналитика и журнала действий;

- разработан механизм декларативных правил YAML с девятью детекторами, включая корреляционный сценарий «перебор — успешный вход — подозрительный процесс»;

- создано десктопное приложение на PySide6 с панелью мониторинга, поиском, управлением правилами и экспортом отчётов HTML/JSON;

- обеспечена поставка в виде исходного кода и автономного исполняемого файла miniSIEM.exe; подготовлена документация по развёртыванию.

Поставленная цель достигнута — прототип закрывает полный цикл событийной аналитики на одной машине: от сырого 4625 в журнале Security до high-алерта во вкладке Alerts и HTML-отчёта по инциденту. На уровне идей он повторяет то, что делают промышленные SIEM в крупных SOC, только в десятки раз проще и без лицензионных платежей.

Для кафедры — готовый стенд: путь события → правило 4625-BURST → алерт → корреляция из трёх событий.

Проект с нуля до .exe сделан самостоятельно; на защите покажу импорт EVTX и срабатывание WIN-CORR.

Дальше можно развивать так: вынос сбора в отдельный агент с поддержкой работы с нескольких хостов, добавление экспорта в syslog и интеграции с Telegram-ботом для алертов, подключение поиска IOC по внешним спискам и расширение набора правил под матрицу MITRE ATT&CK как минимум базовые техники Credential Access и Defense Evasion. Собиратель и хранилище уже разделены — дальше наращивать проще уже разделены.

В целом mini SIEM иллюстрирует простой тезис: понимание принципов работы SOC не обязательно требует приобретения Splunk за миллион. Достаточно одного ноутбука, языка Python и готовности изучить журналы

Windows. Дальнейшее развитие проекта предполагается в практических задачах по информационной безопасности после защиты диплома.

Теоретическая значимость работы состоит в систематизации подходов к нормализации журналов Windows, параметризации правил детектирования и организации рабочего места аналитика на конечной точке. Практическая значимость – в готовом прототипе, документации и воспроизводимых сценариях тестирования для специальности «Системы информационной безопасности».

Направления дальнейшего развития включают: агентскую схему сбора с нескольких хостов; централизованное хранилище PostgreSQL или OpenSearch; потоковую шину Kafka; интеграцию с SOC–процессами оператора связи; расширение библиотеки детекторов по матрице MITRE ATT&CK; доставку алертов во внешние каналы [11].

На защите: импорт EVTX → фильтр 4625 → алерт → статус + комментарий → HTML. Укладываюсь в 7–10 минут.

Выполнены все задачи, сформулированные во введении: проведён анализ предметной области и существующих SIEM; составлено техническое задание; обоснован и применён стек Python/SQLite/PySide6/YAML; спроектирована и сделана модульная архитектура; проведено тестирование с подтверждением критериев приёмки.

Разработанный mini SIEM может служить учебным стендом кафедры «Кибербезопасности и искусственного интеллекта» для демонстрации основ событийной аналитики Windows. Исходный код размещён в репозитории проекта с документацией по развёртыванию.

Дальнейшее развитие mini SIEM может включать интеграцию с SOC–процессами оператора АО «ТрансТелеКом» при сохранении локального учебного контура как лабораторного стенда кафедры [1].

Список использованной литературы

1. Официальный сайт АО «ТрансТелеКом». URL: <https://www.ttk.ru/> дата обращения: 11.05.2026.
2. NIST SP 800–92. Guide to Computer Security Log Management. URL: <https://csrc.nist.gov/publications/detail/sp/800–92/final> дата обращения: 25.05.2026.
3. Microsoft. Windows Security auditing. URL: <https://learn.microsoft.com/windows/security/threat-protection/auditing/> дата обращения: 11.05.2026.
4. Гостев В.А. Информационная безопасность: учебник для вузов. – М.: Юрайт, 2022.
5. Олифер В.Г., Олифер Н.А. Компьютерные сети. Принципы, технологии, протоколы: учебник для вузов. – СПб.: Питер, 2022.
6. Баранов А.В. Телекоммуникационные системы и сети: учебное пособие. – СПб.: Питер, 2021.
7. Девянин П.Н. Модели безопасности компьютерных систем. – М.: Горячая линия – Телеком, 2020.
8. Шаньгин В.Ф. Информационная безопасность компьютерных систем и сетей: учебное пособие. – М.: Форум: ИНФРА–М, 2017.
9. ITU. Материалы Международного союза электросвязи по телекоммуникациям. URL: <https://www.itu.int/> дата обращения: 11.05.2026.
10. RFC 5424: The Syslog Protocol. – IETF, 2009. URL: <https://www.rfc-editor.org/rfc/rfc5424> дата обращения: 11.05.2026.
11. MITRE ATT&CK Framework. URL: <https://attack.mitre.org/> дата обращения: 11.05.2026.
12. Мельников В.П., Петраков А.М. Информационная безопасность: учебное пособие. – М.: Академия, 2021.
13. ISO/IEC 27001:2022. Information security management systems – Requirements. – Geneva: ISO, 2022.
14. ГОСТ Р ИСО/МЭК 27001–2021. Информационная технология. Методы и средства обеспечения безопасности. Системы менеджмента информационной безопасности. Требования. – М.: Стандартинформ, 2021.
15. Microsoft. Sysinternals Sysmon. URL: <https://learn.microsoft.com/sysinternals/downloads/sysmon> дата обращения: 11.05.2026.
16. Щербakov А.Ю. Введение в системы обнаружения вторжений. – М.: Гелиос АРВ, 2019.
17. Wazuh. The Open Source Security Platform: Documentation. URL: <https://documentation.wazuh.com/> дата обращения: 18.05.2026.
18. Elastic. Elastic Stack Documentation. URL: <https://www.elastic.co/guide/index.html> дата обращения: 18.05.2026.
19. ENISA. Публикации по кибербезопасности. URL: <https://www.enisa.europa.eu/publications> дата обращения: 11.05.2026.

20. Python Software Foundation. Python 3.11 Documentation. URL: <https://docs.python.org/3.11/> дата обращения: 25.05.2026.
21. pywin32: Python for Win32 extensions: Documentation. URL: <https://github.com/mhammond/pywin32> дата обращения: 25.05.2026.
22. SQLite Documentation. URL: <https://www.sqlite.org/docs.html> дата обращения: 11.05.2026.
23. PyYAML Documentation. URL: <https://pyyaml.org/wiki/PyYAMLDocumentation> дата обращения: 25.05.2026.
24. Qt for Python PySide6 Documentation. URL: <https://doc.qt.io/qtforpython/> дата обращения: 25.05.2026.
25. PyInstaller Manual. URL: <https://pyinstaller.org/> дата обращения: 25.05.2026.